

Unit-1

Introduction to Operating System





Outline

- Computer system overview
- Computer system architecture
- What is Operating System (OS)
- Roles of Operating System (OS)
- Objectives / Goals of Operating System (OS)
- Generations of Operating Systems (OS)
- Operating Systems (OS) services
- Types of Operating Systems (OS)
- System calls
- Operating Systems (OS) structure
- Multiprogramming v/s Multiprocessing v/s Multitasking
- Time Sharing Operating System
- Parallel Processing Operating System
- Distributed Operating System



Computer system overview

Section - 1

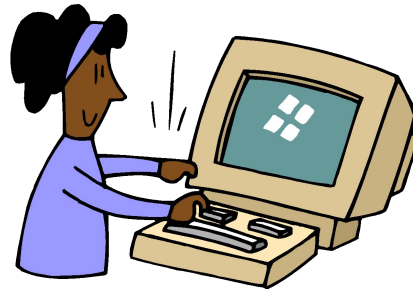
Basic elements of computer

Processor

**I/O
Modules**

Memory

**System
Bus**



Processor

Referred to as the
Central Processing Unit (CPU)



Arithmetic & Logic Unit

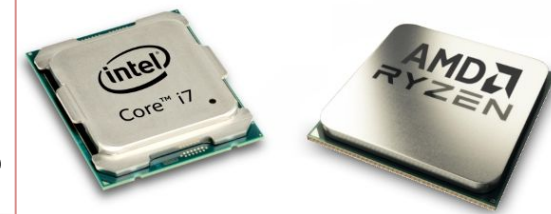
Performs the **data processing** functions

Control Unit

Controls the operation of the computer

Exercise

Which two companies are famous for manufacturing computer processors?



Memory

Memory is device that is used to store data/information



Primary Memory

- Volatile
- RAM & ROM

Secondary Memory

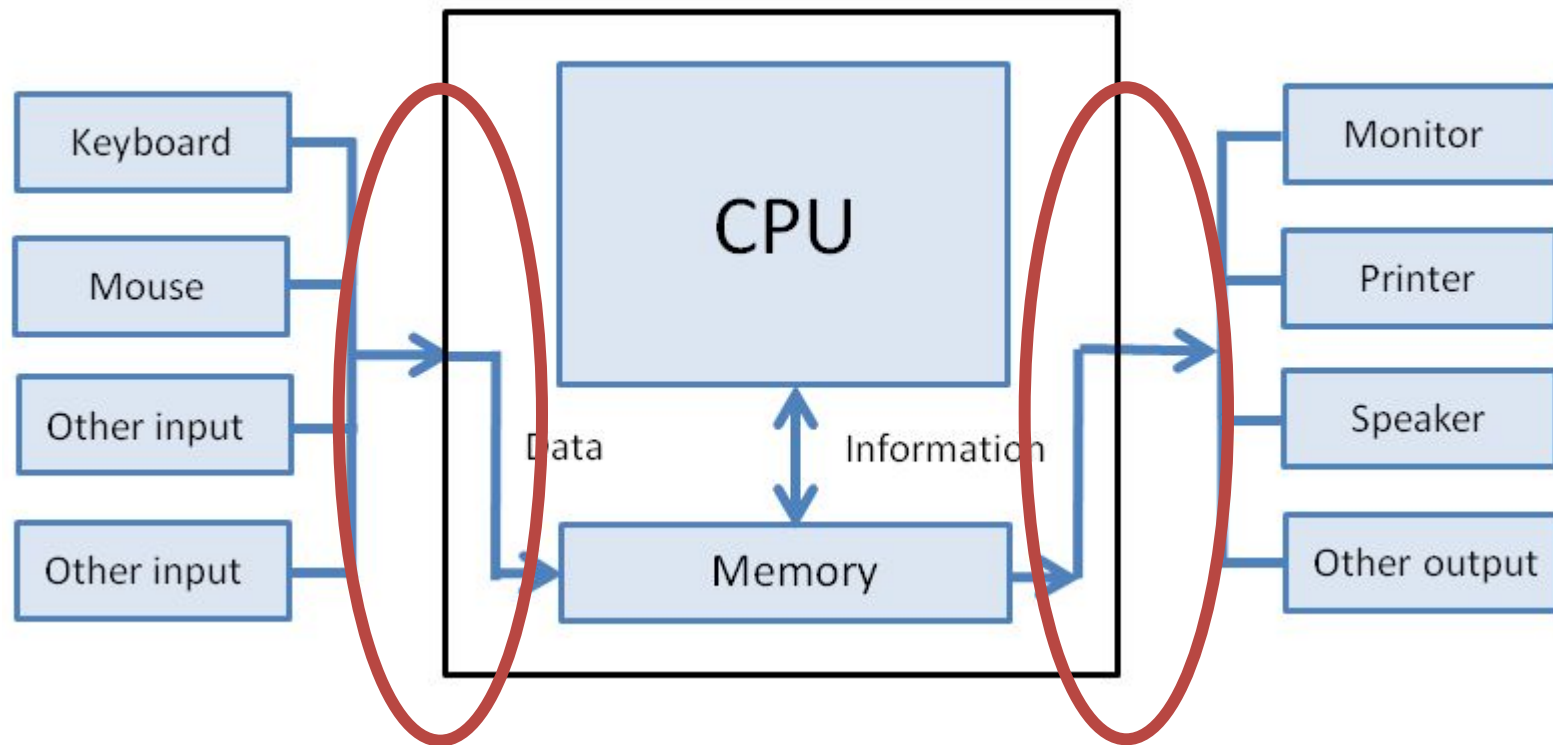
- Non-Volatile
- HDD, CD & DVD

Give the difference between primary memory and secondary memory.

Exercise

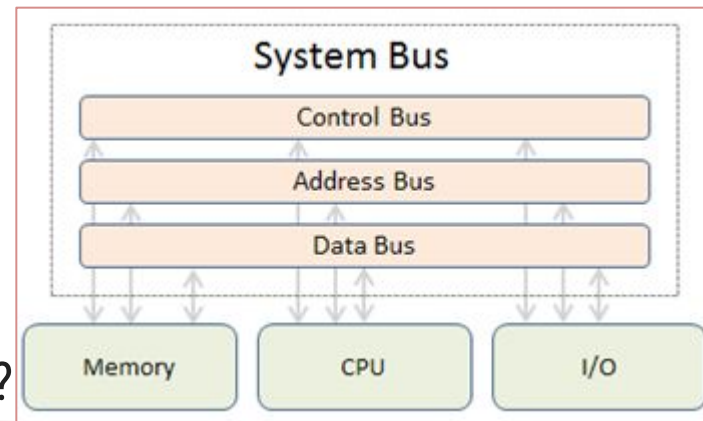
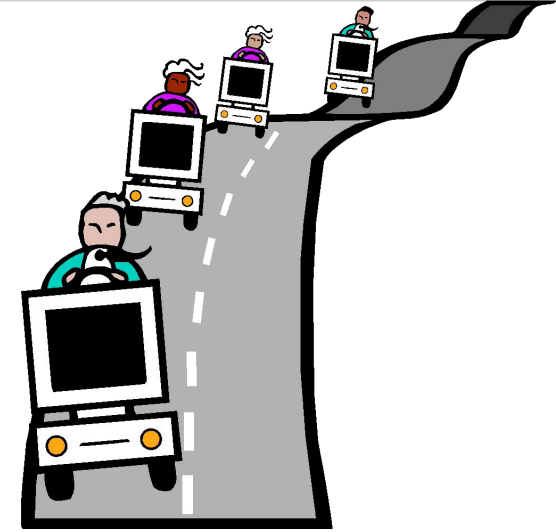
I/O Module (Input/Output Module)

- Input/output module is a **device that acts as the connective bridge between a computer system** at one end **and an I/O or peripheral device** at the other, such as a printer, webcam or scanner.,



System Bus

- ❑ Provides communication among processors, main memory, and I/O devices.
- ❑ The system bus is a pathway **composed of cables and connectors used to carry data** between a computer microprocessor and the main memory.
- ❑ Types of buses
 - ❑ Address bus - **carries memory addresses** from the processor to other components such as primary storage and input/output devices.
 - ❑ Data bus - **carries the data** between the processor and other components.
 - ❑ Control bus - **carries control signals** from the processor to other components.



What are the 3 types of buses?

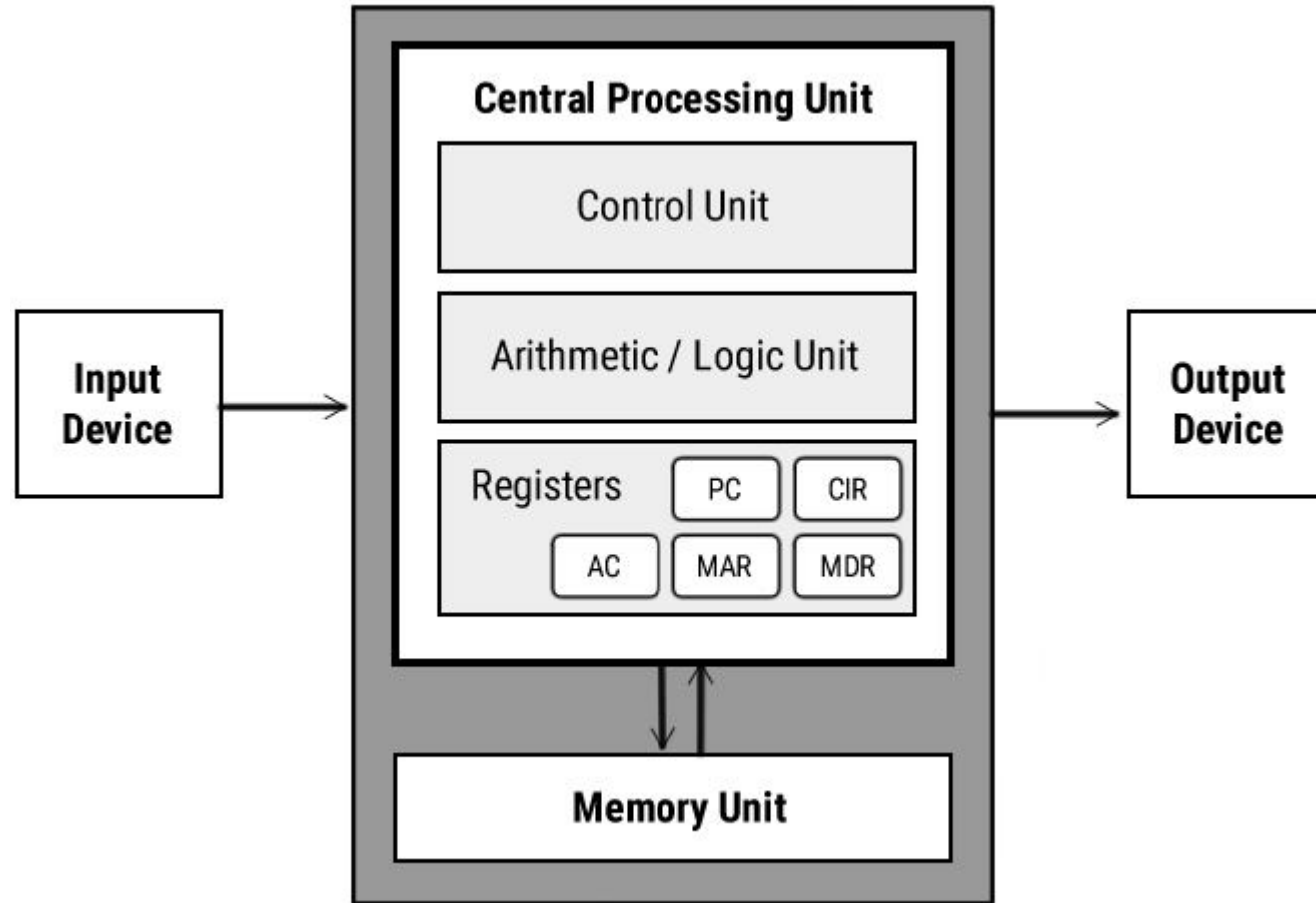
Exercise



Computer system architecture

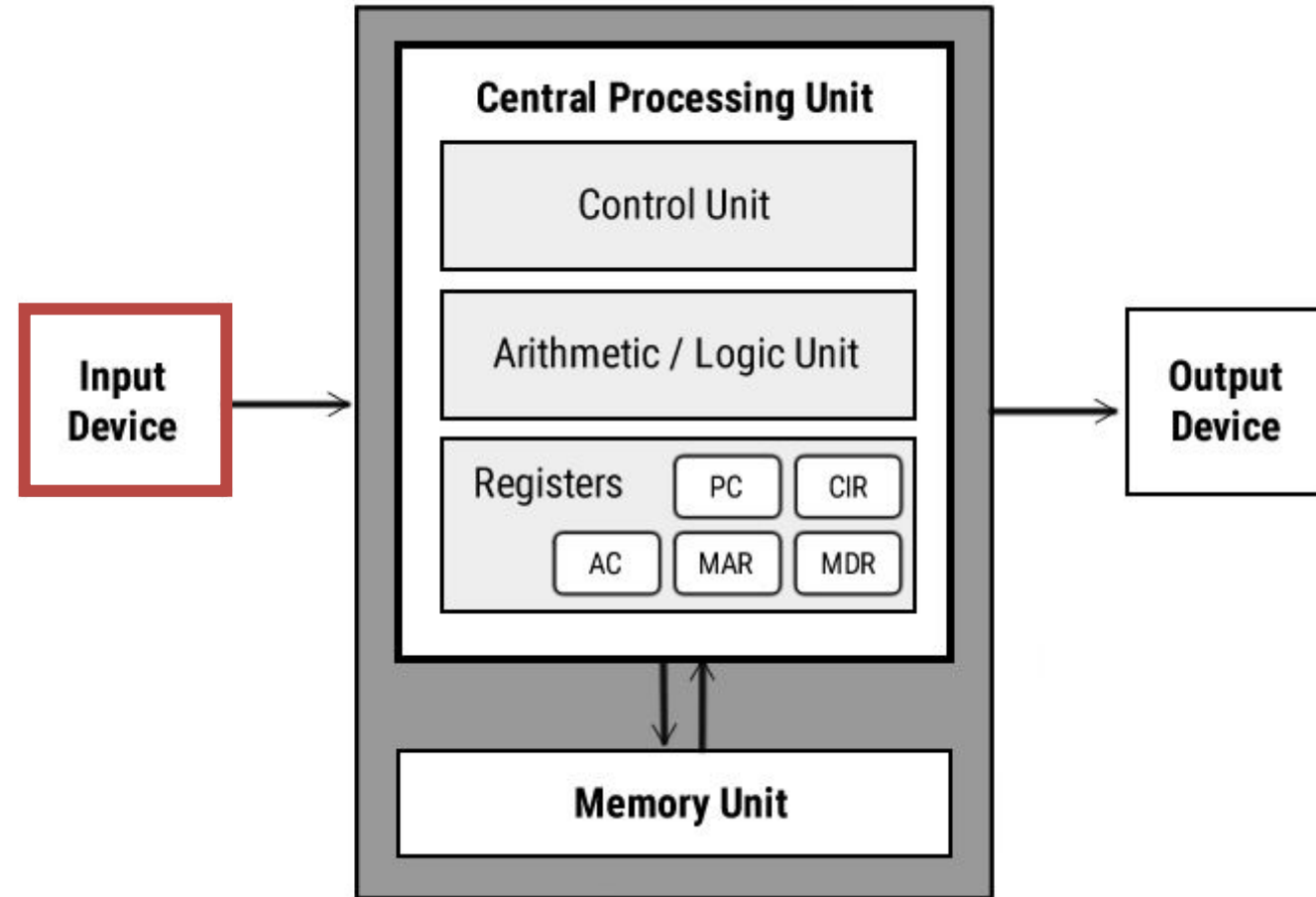
Section - 2

Computer system architecture



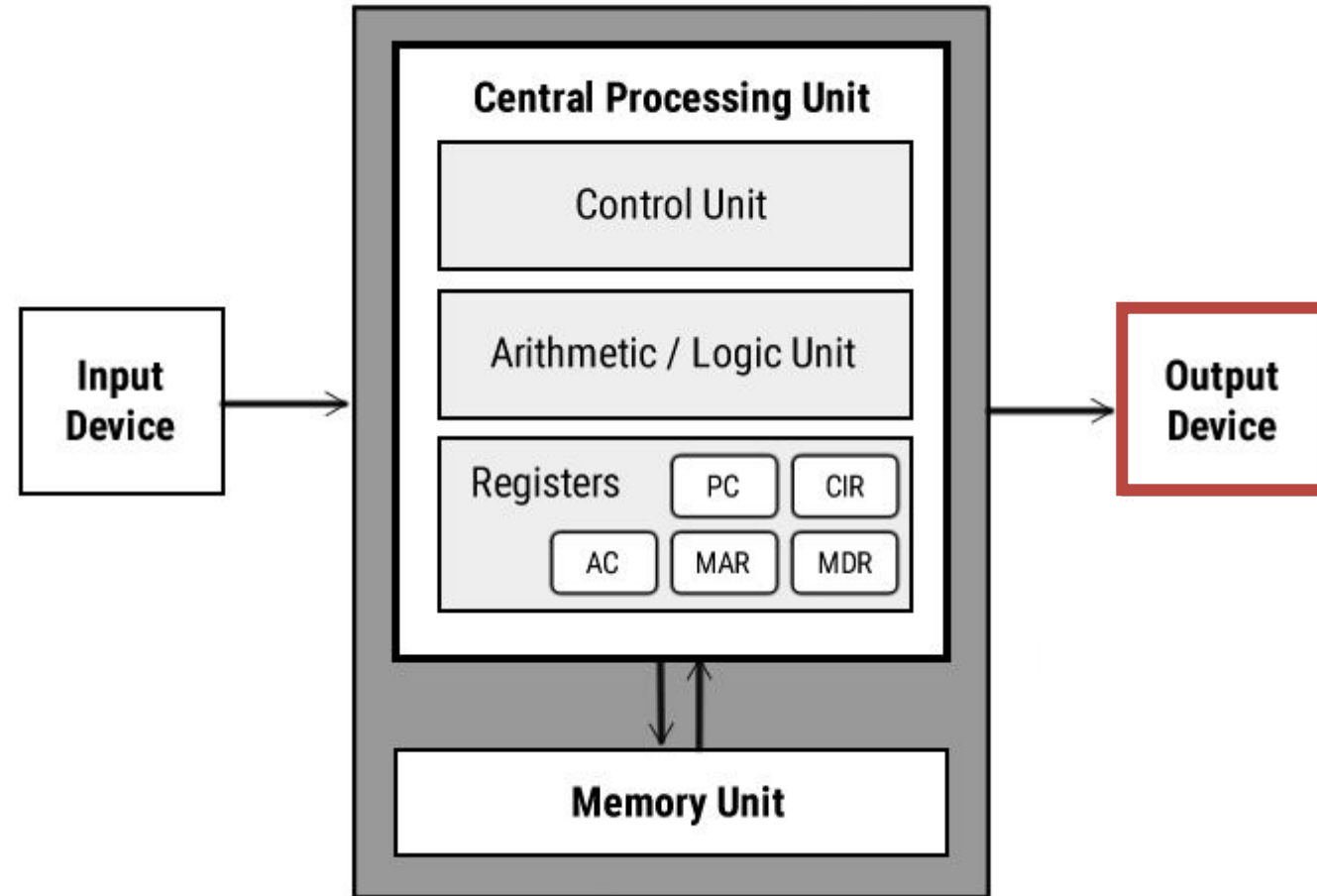
Input unit

- ❑ It **provides data** and **instructions** to the computer system.
- ❑ Commonly used input devices are keyboard, mouse, magnetic tape etc.
- ❑ Input unit performs following tasks:
 - ❑ **Accept the data** and **instructions** from the outside environment.
 - ❑ **Convert** it into **machine language**.
 - ❑ **Supply the converted data** to computer system.



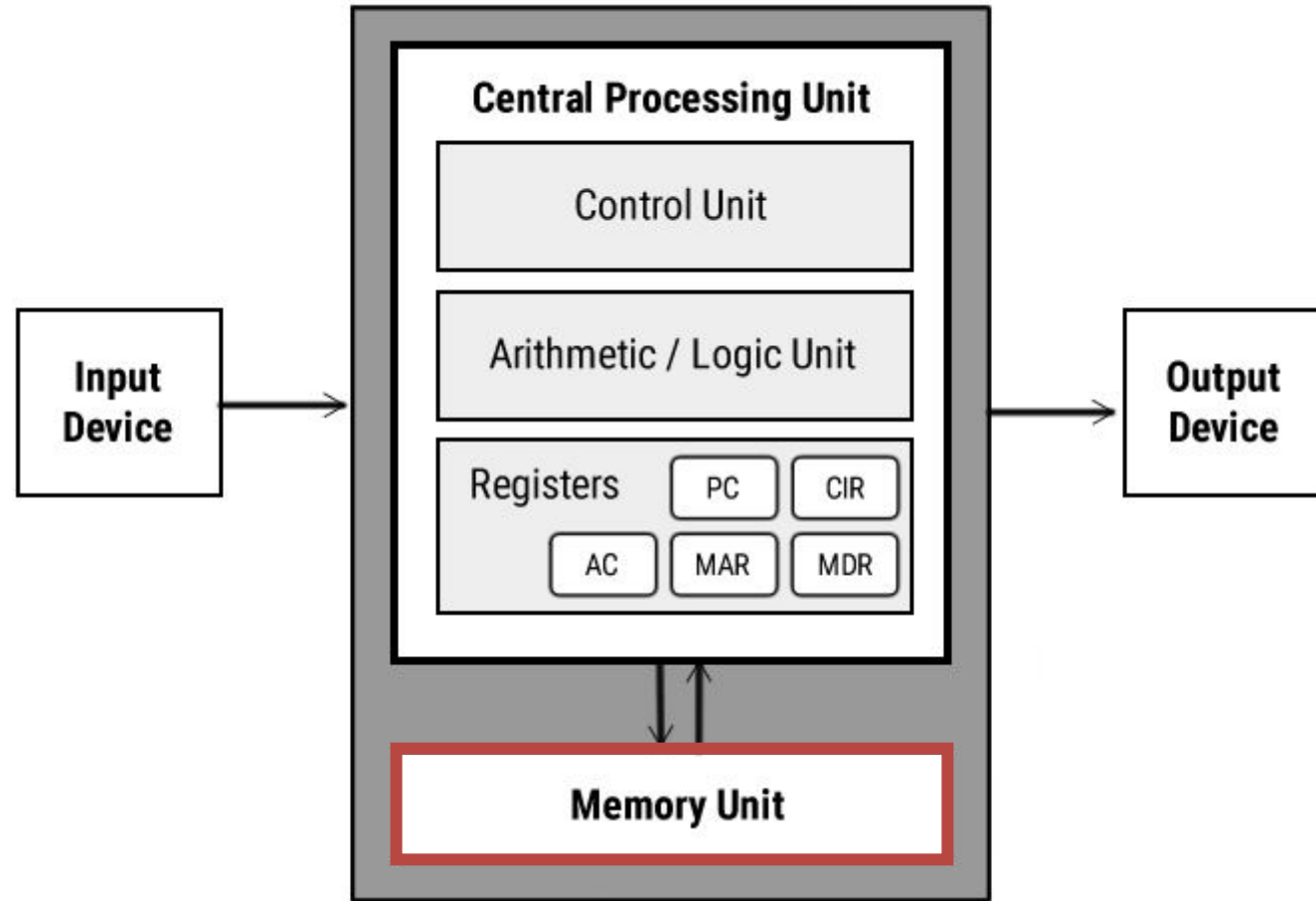
Output unit

- ❑ It **connects the internal system** of a computer **to the external environment**.
- ❑ It **provides the results** of any computation, or instructions to the outside world.
- ❑ Some output devices are printers, monitor etc.



Storage unit

- This unit **holds the data** and **instructions**.
- It also **stores the intermediate results** before these are sent to the output devices.
- It also **stores the data for later use**.
- The storage unit of a computer system can be divided into two categories:
 - Primary Storage
 - Secondary Storage

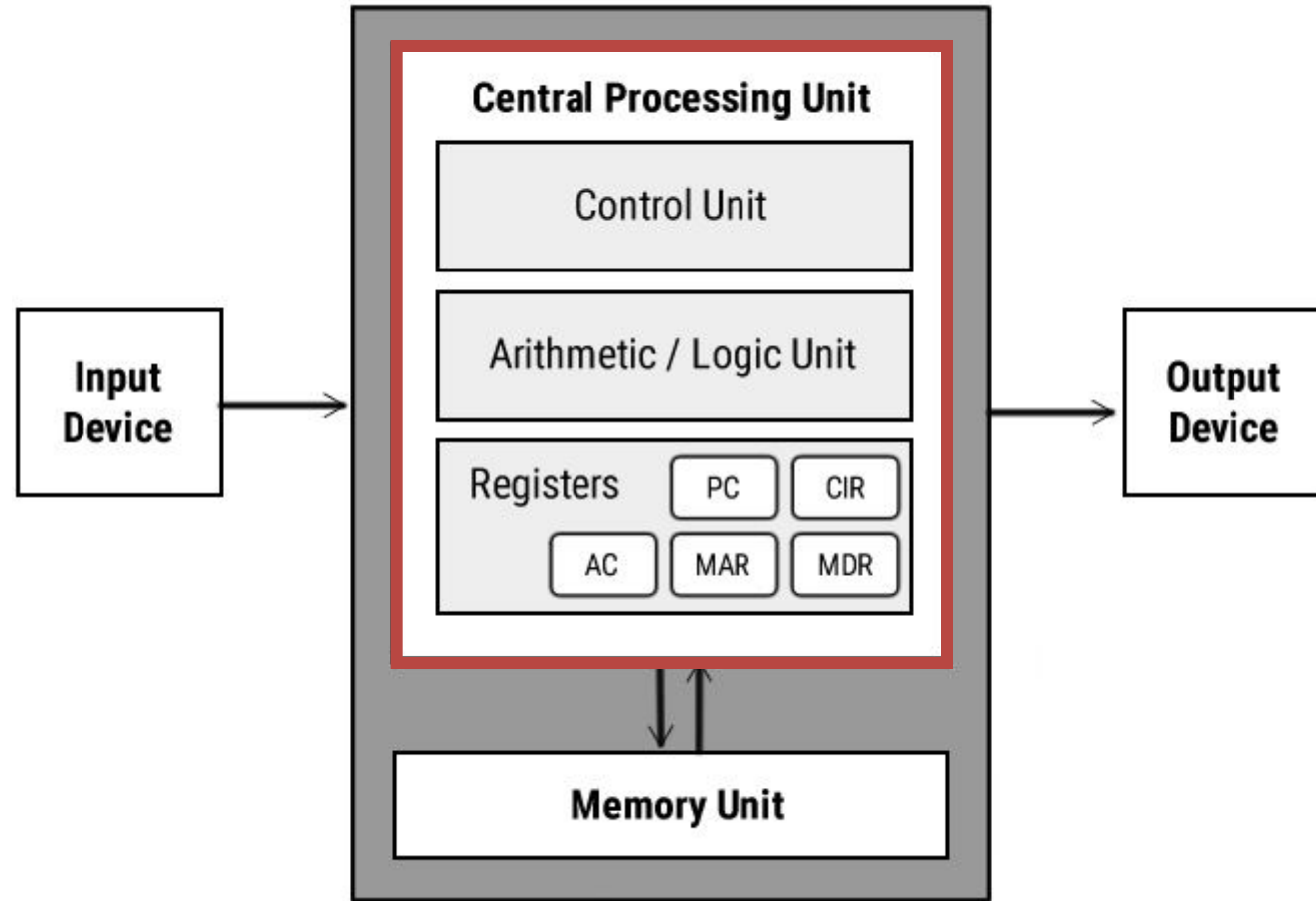


Primary storage (memory) vs Secondary storage (memory)

Primary storage	Secondary storage
Examples: RAM, ROM, Cache memory, PROM, EPROM, Registers, etc.	Examples: Hard Disk, Floppy Disk, Magnetic Tapes, etc.
It is temporary and volatile.	It is permanent and Non-volatile.
Primary memory is directly accessible by Processor/CPU.	Secondary memory is not directly accessible by the CPU.
Primary memory devices are expensive.	Secondary memory devices are cheaper.
The memory devices used for primary memory are semiconductor memories.	The secondary memory devices are magnetic and optical memories.
Primary memory is also known as Main memory or Internal memory.	Secondary memory is also known as External memory or Auxiliary memory.

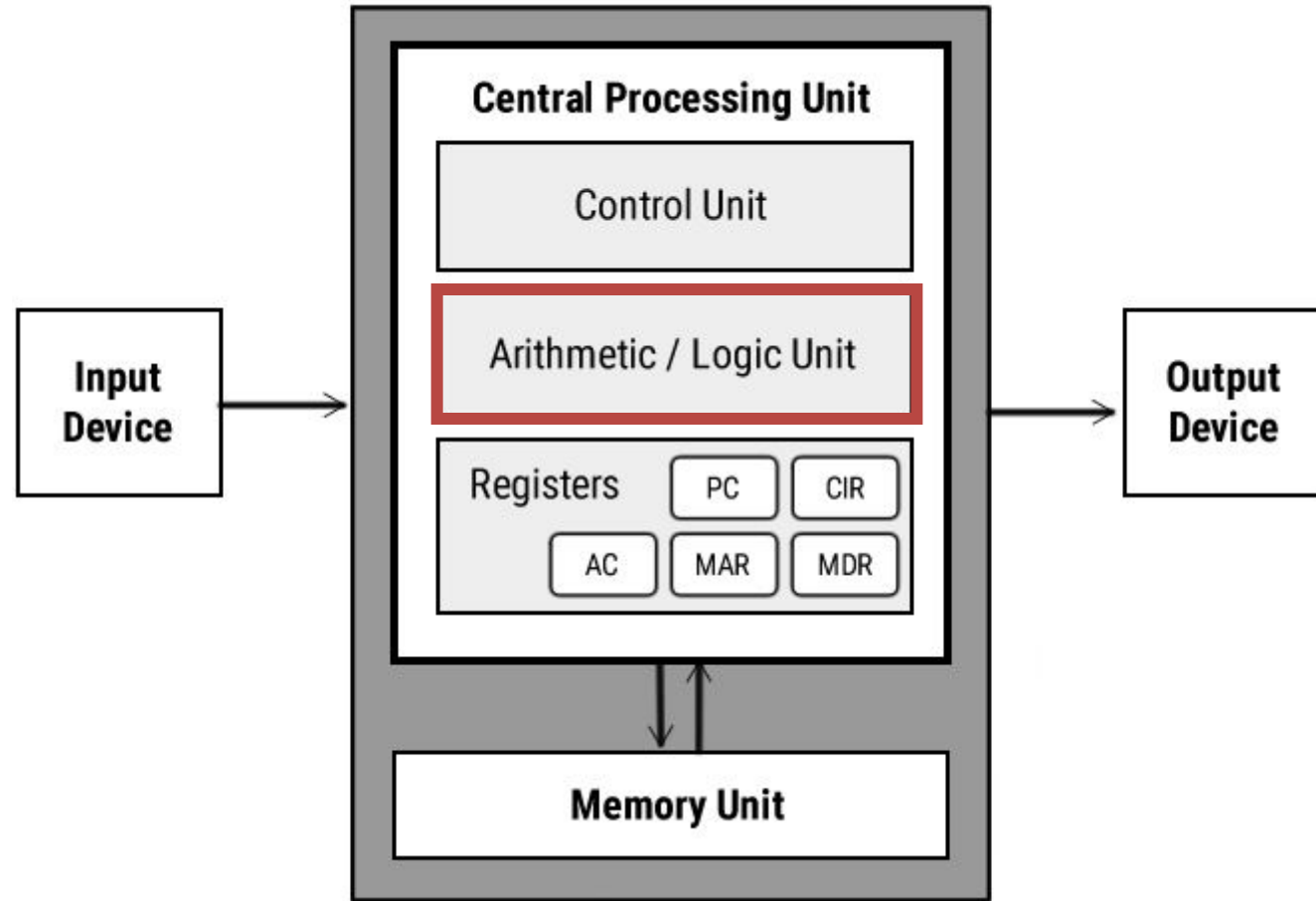
CPU (Central Processing Unit)

- ❑ The **Arithmetic Logical Unit and Control Unit** are together known as CPU.
- ❑ CPU is the **brain** of computer system.
- ❑ It performs following tasks:
 - ❑ performs all operations.
 - ❑ takes all decisions.
 - ❑ controls all the units of computer.



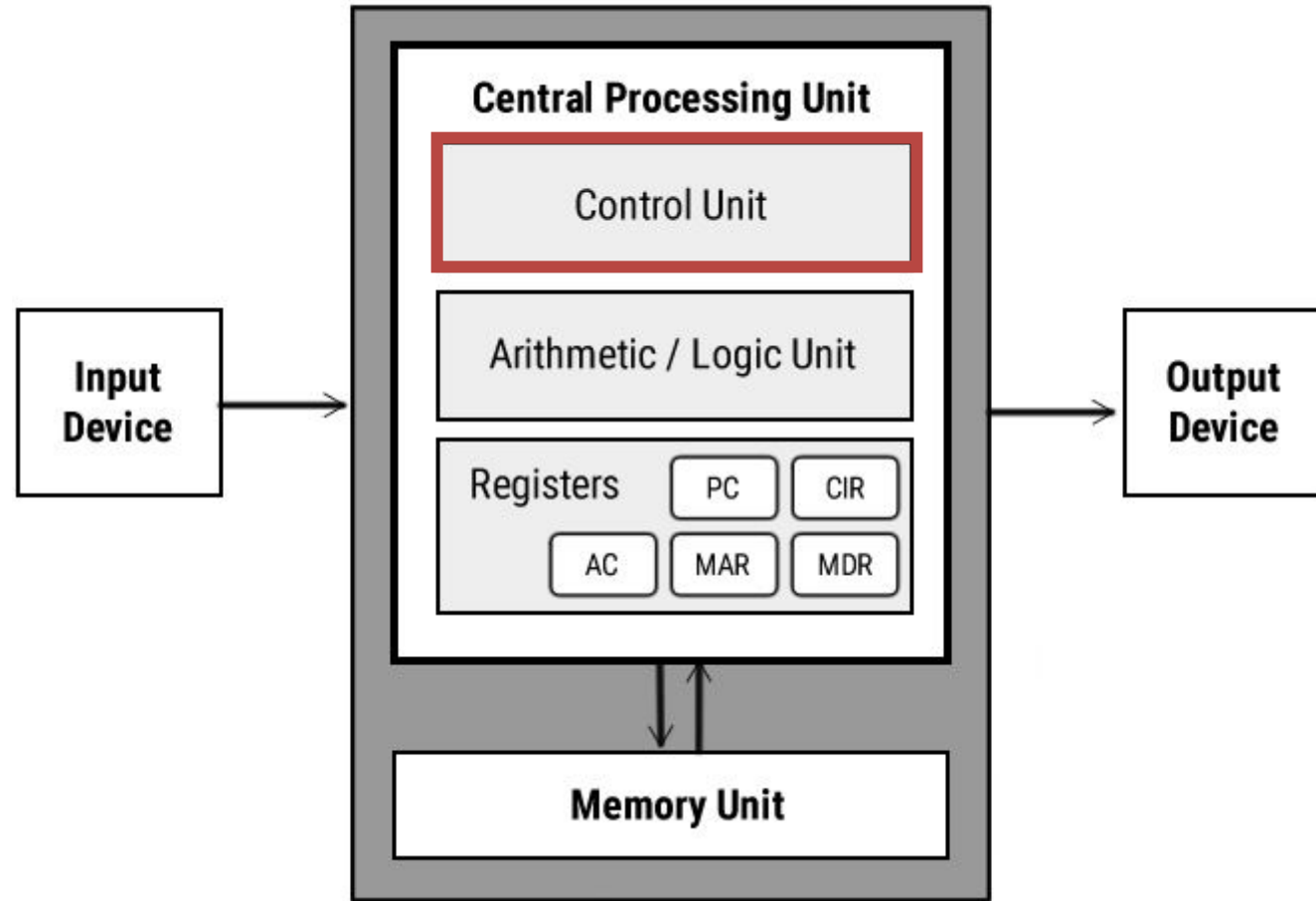
ALU (Arithmetic Logical Unit)

- ❑ All the **calculations** are performed in ALU of the computer system.
- ❑ The ALU can **perform basic operations** such as addition, subtraction, division, multiplication etc.
- ❑ Whenever calculations are required, the **control unit transfers the data from storage unit to ALU**.
- ❑ When the operations are done, the result is transferred back to the storage unit.



CU (Control Unit)

- It **controls all other units** of the computer.
- It **controls the flow of data and instructions** to and from the storage unit to ALU.
- Thus it is also known as central nervous system of the computer.





What is Operating System (OS)

Section - 3

What is Operating System (OS)?

□ A Computer System consists of various hardware's such as



Processor



RAM



Keyboard & Mouse



Hard Disk



Monitor



Printer

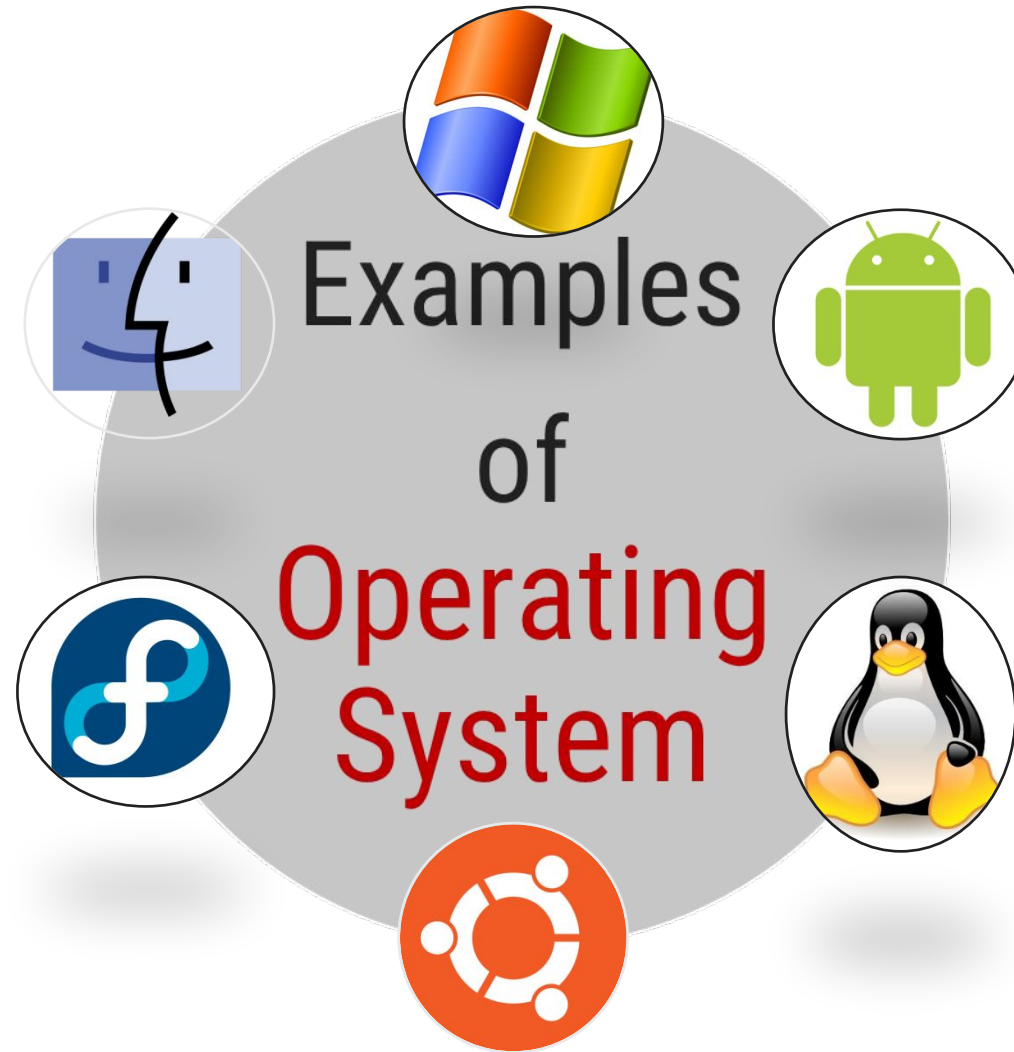
Who manages (controls) these hardware's???

Operating System

Definition of Operating System (OS)

- An Operating System (OS) is a **collection of software** that
 - **manages hardware resources**
 - **provides various service** to the users

Examples of Operating System (OS)



Where OS lies? (Interaction of OS & Hardware)

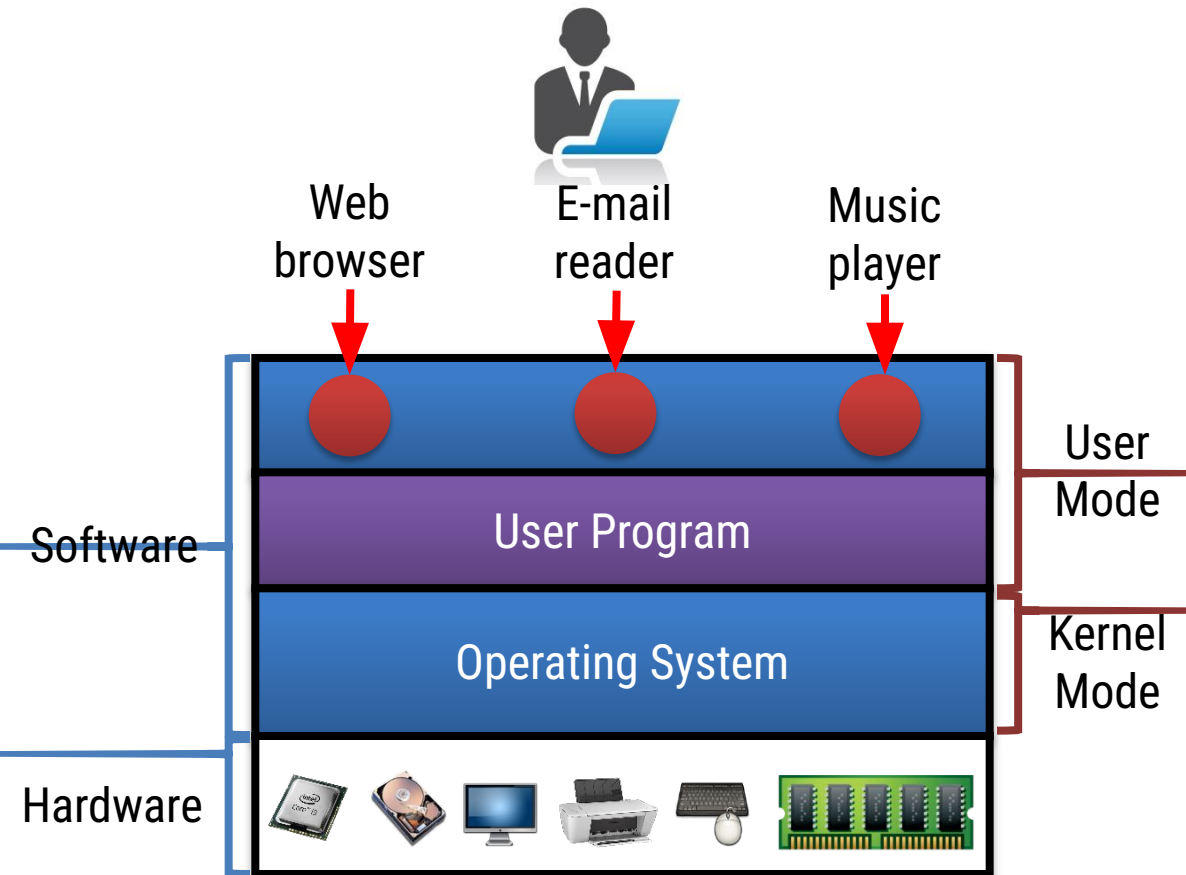
- OS **lies between hardware and user program.**
- It **acts as an intermediary** between the user and the hardware.
- Modes of operation of computer

1. Kernel Mode

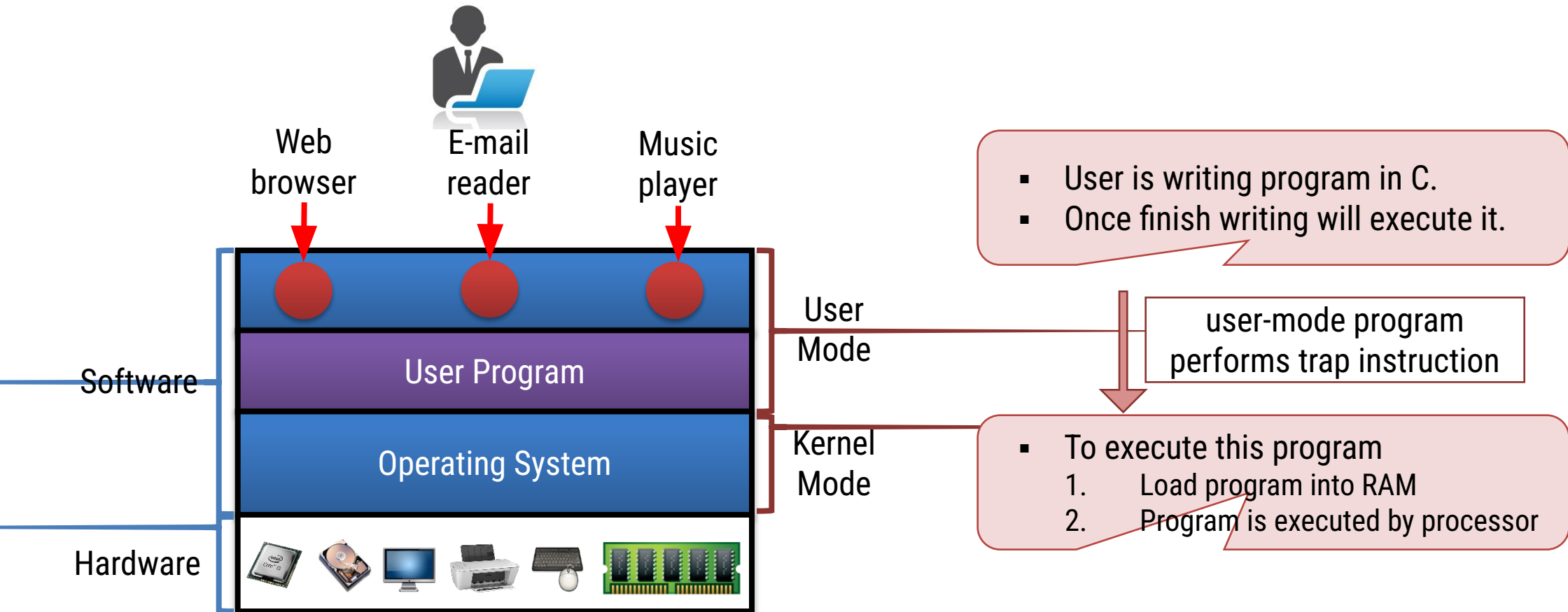
- has **complete access** to all the hardware
- can **execute any instruction** that a machine is capable of executing
- has **high privileged** (rights)

2. User Mode

- has **limited access** to limited hardware
- can **execute only subset (few)** of the machine instructions
- has **less privileged** (rights)



Why and How switch occur?





Roles of Operating System (OS)

Section - 4

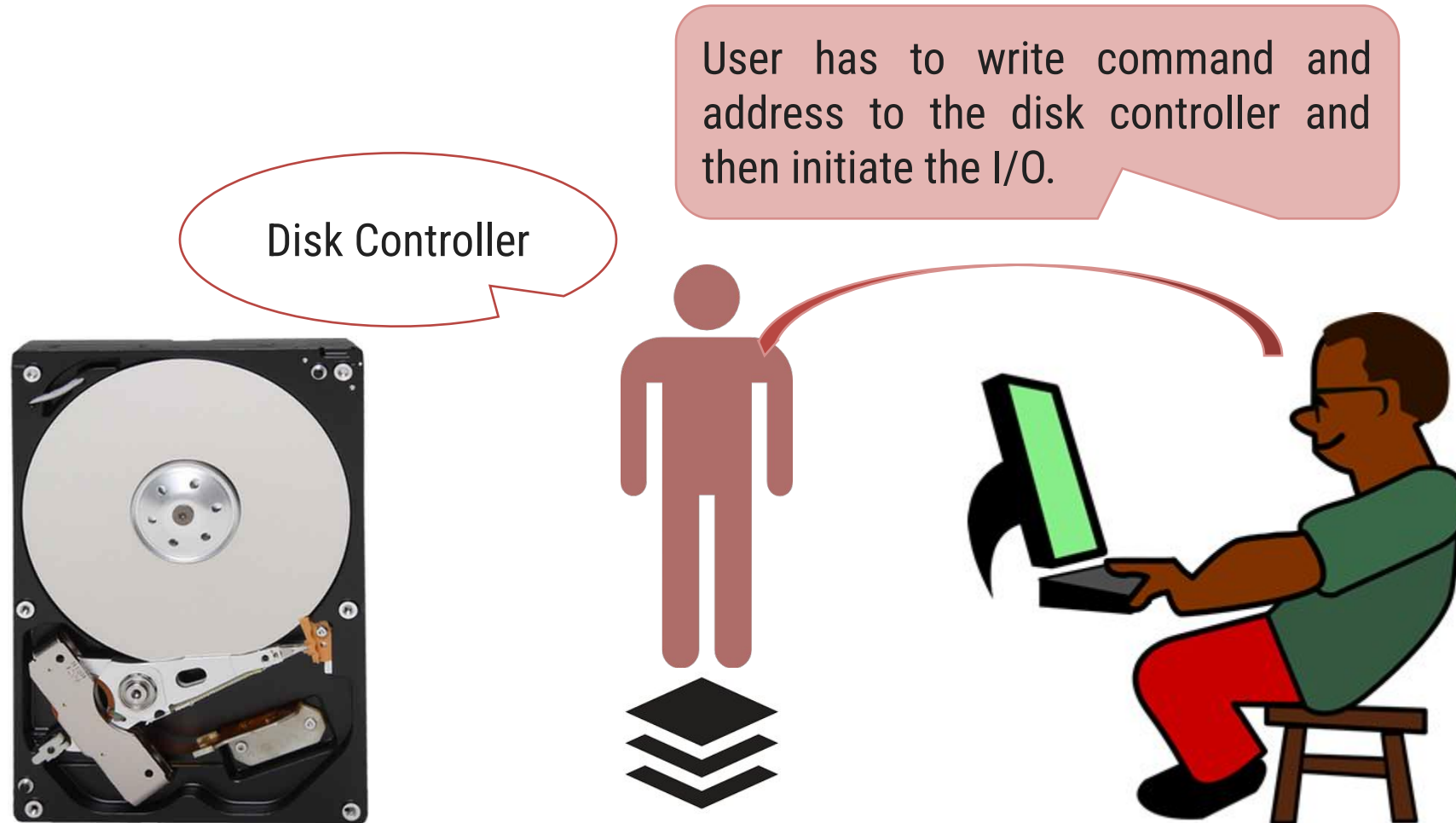
OS as Extended Machine

- The architecture of a computer is difficult to program
 - Architecture (instruction set, memory organization, I/O, bus structure) of most of computer at the machine level language is primitive and awkward to program.
 - Example: If **user want to read from floppy or hard disk**:



OS as Extended Machine

□ Example: If **user want to read from floppy or hard disk:**



OS as Extended Machine

The disk controller will find the requested data in the disk and fetch it from disk to disk controller buffer.

User has to check the status of disk controller operation where it has finished or not.



OS as Extended Machine

If success, the data from disk controller buffer should be moved to main memory (to the application buffer).



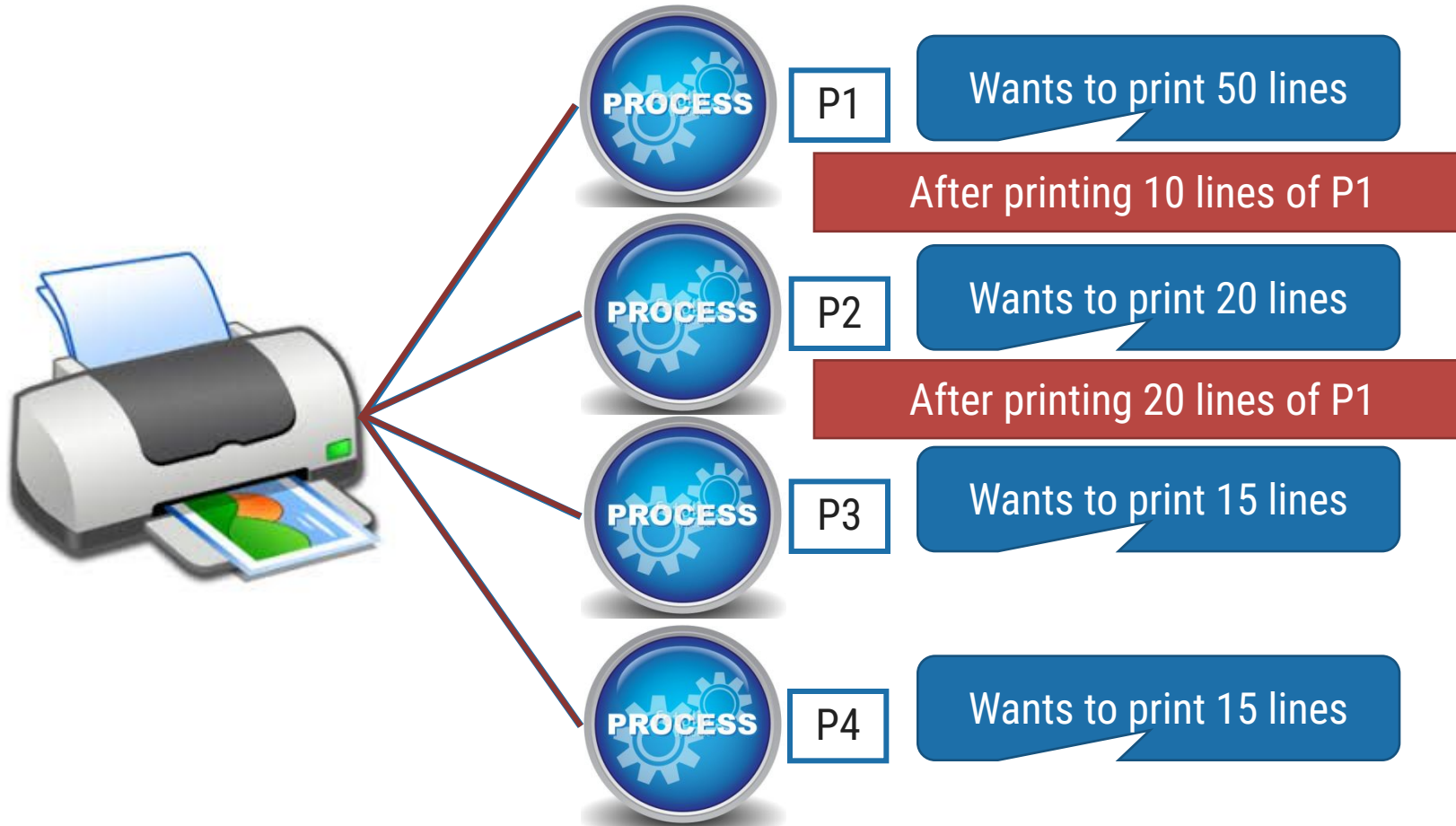
OS as Resource Manager

- There are lots of resources in computer system
 - CPU (Processor)
 - Memory
 - I/O devices such as hard disk, mouse, keyboard, printer, scanner etc.
- If a computer system is used by **multiple applications (or users)**, then they will **compete for these resources**.



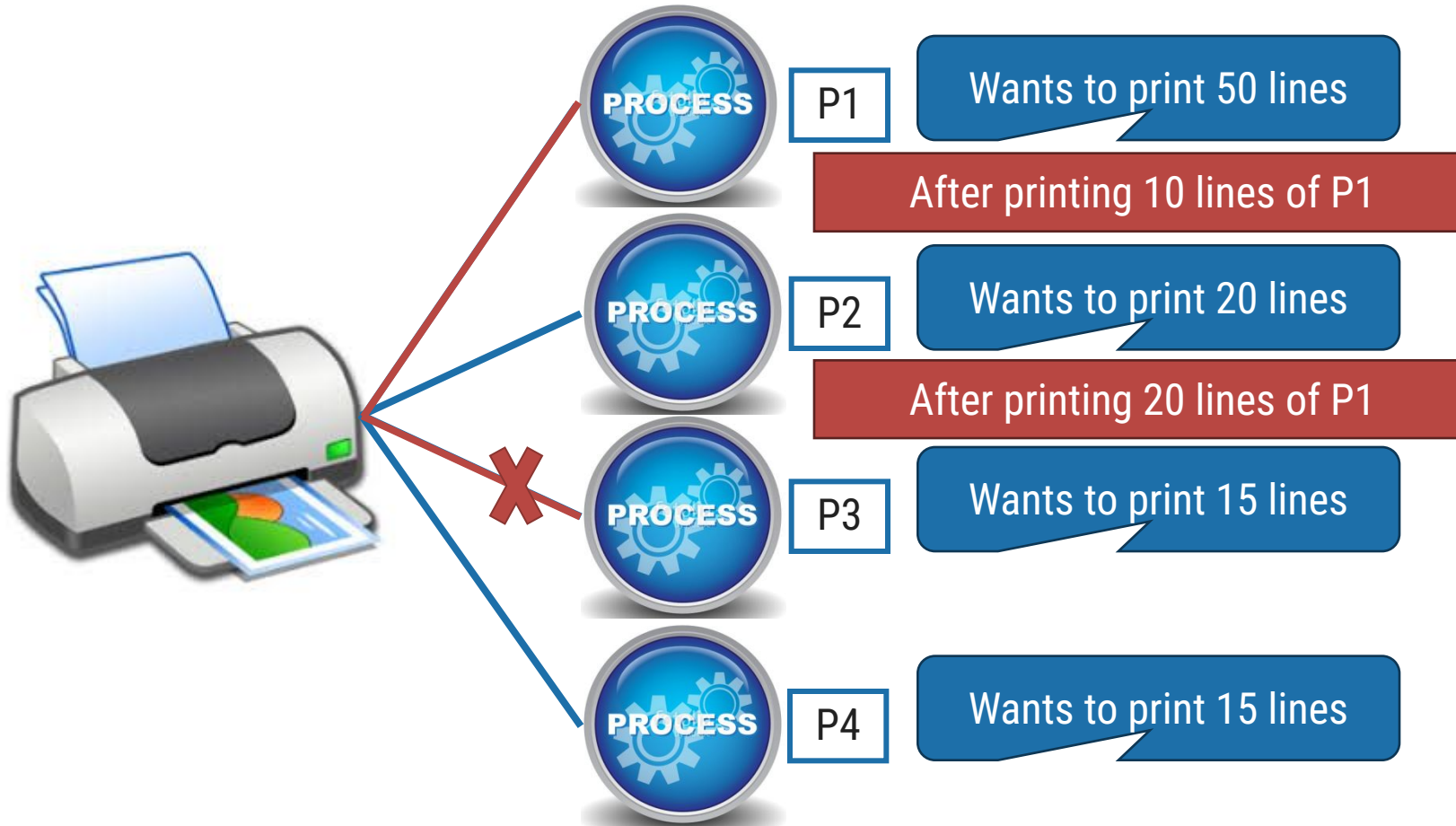
OS as Resource Manager

- It is the job of OS to allocate these resources to the various applications so that:
 - The **resources are allocated fairly** (equally)



OS as Resource Manager

- It is the job of OS to allocate these resources to the various applications so that:
 - The **resources are protected from cross-access**.



OS as Resource Manager

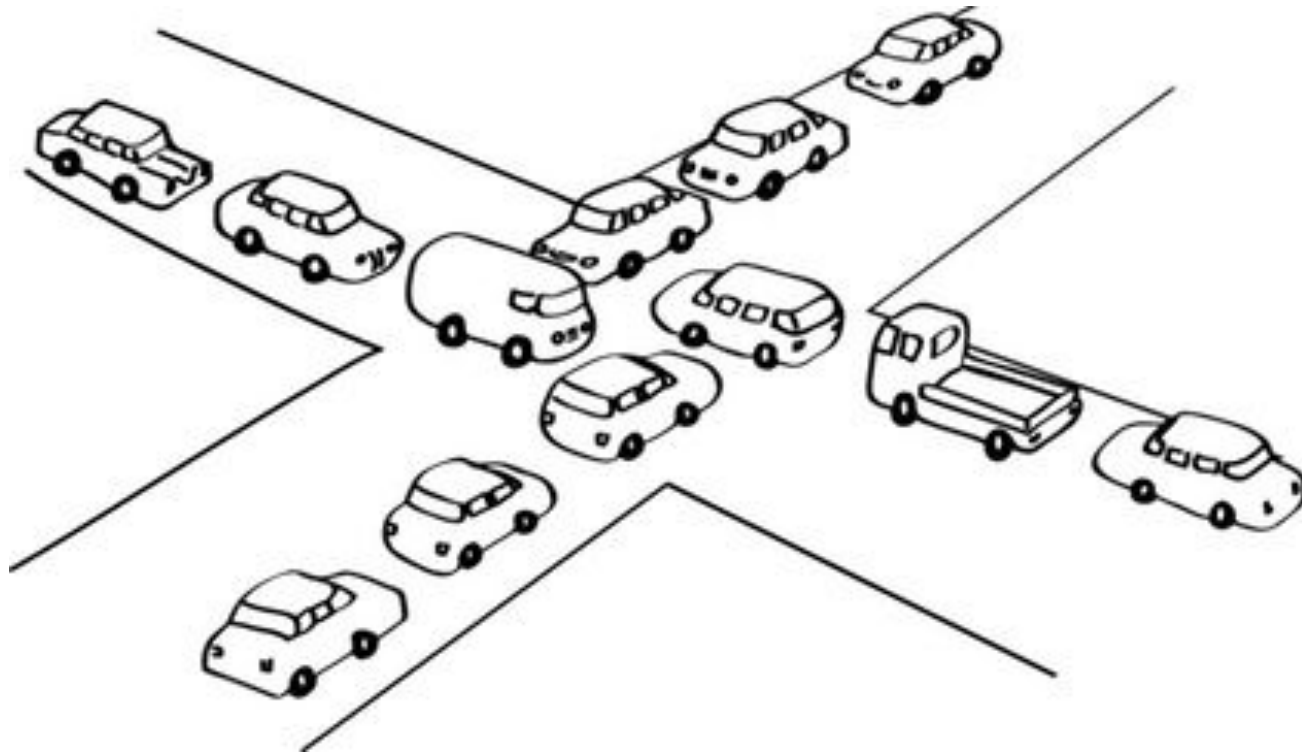
- It is the job of OS to allocate these resources to the various applications so that:
 - **Access to the resources is synchronized** so that operations are correct and consistent
 - Example: If we write a program to calculate below in C language

$$7 + 9 - 6 * 4 / 2 = 4$$

The diagram illustrates a resource allocation scenario. A horizontal red line represents a resource pool. Four blue circles, labeled P1, P2, P3, and P4, are positioned above the line. Below each circle is a black number representing the resource count for that process: 3 for P1, 4 for P2, 1 for P3, and 2 for P4. A red arrow points from the right side of the resource pool to the number 20, which is followed by a large red 'X'.

OS as Resource Manager

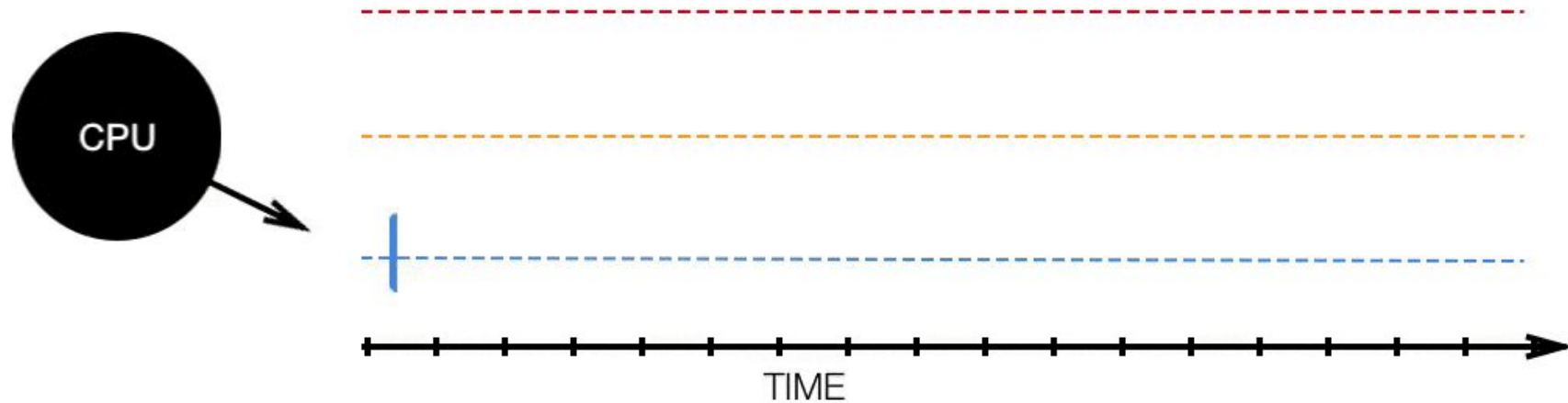
- It is the job of OS to **proper allocate these resources to the various applications** so that:
 - Deadlock are detected, resolved and avoided.



OS as Resource Manager

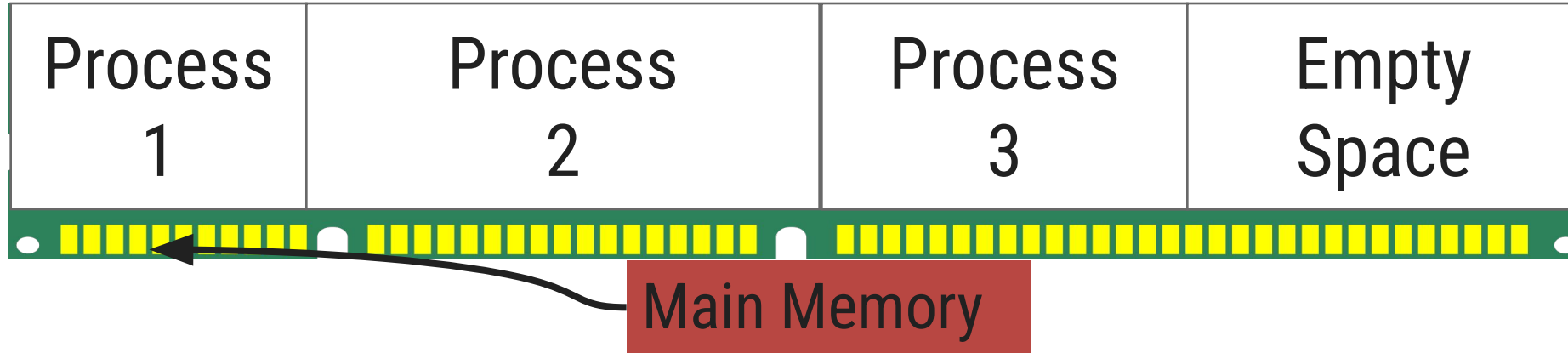
□ Resource manager – sharing resources in two different ways:

1. In **time sharing/multiplexing** (i.e **CPU**)



OS as Resource Manager

- Resource manager – sharing resources in two different ways
 1. In **time sharing/multiplexing**. (i.e **Processor**)
 2. In **space sharing/multiplexing**. (i.e **Memory**)





Objectives / Goals of Operating System (OS)

Section - 5

Objectives / Goals of Operating System (OS)

- ❑ Make the computer system convenient to use in an **efficient manner**.
- ❑ **Hide the details of the hardware** resources from the users.
- ❑ **Provide** users a convenient **interface** to use the computer system.
- ❑ **Act as an intermediary** between the **hardware and its users**, making it easier for the users to access and use other resources.
- ❑ **Manage the resources** of a computer system.
- ❑ **Provide efficient** and **fair sharing of resources** among users and programs.



Operating Systems (OS) services

Section - 7

Services / Functions / Tasks of Operating System (OS)

Definition of Operating System

- An Operating System (OS) is a collection of software that
 - manages hardware resources
 - provides various service to the users

Which?

Services / Functions / Tasks of Operating System (OS)

1. Program development

- It **provides editors and debuggers** to assist (help) the programmer in creating programs.



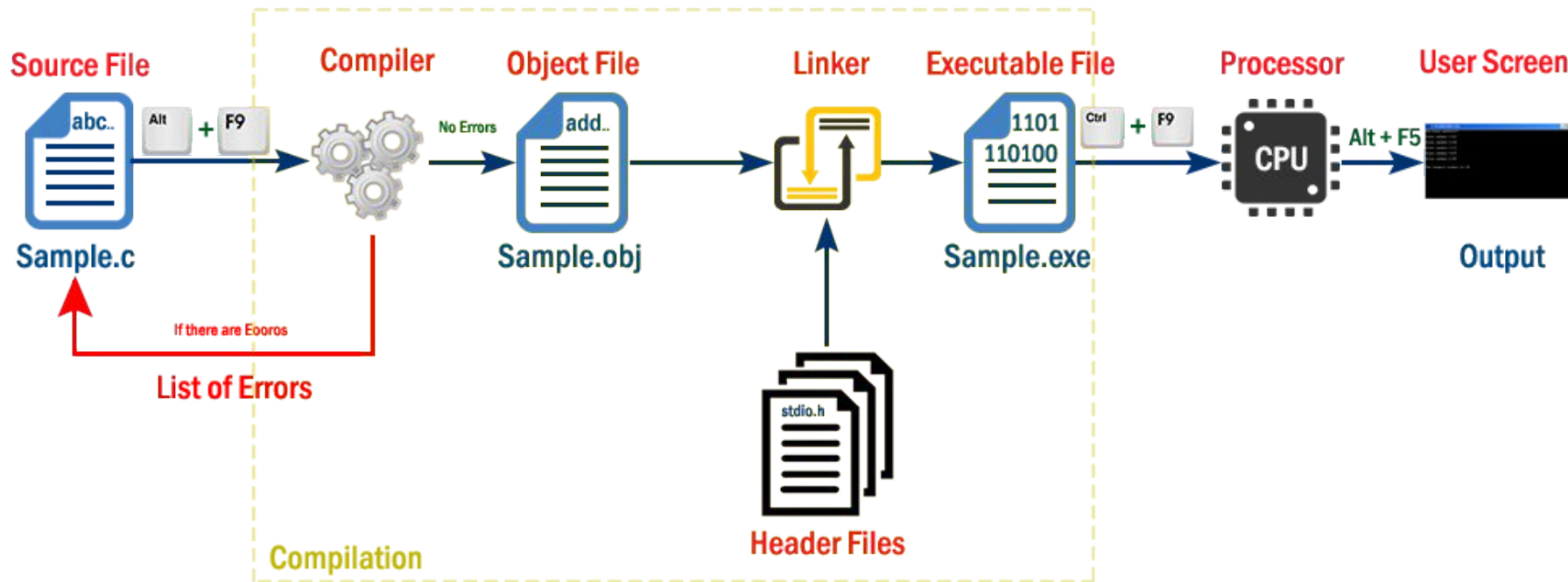
Exercise

Give the name of any five code editor for windows?

Services / Functions / Tasks of Operating System (OS)

2. Program execution

- Following tasks need to be performed to execute a program:
 - Instructions and data must be loaded into main memory.
 - I/O devices and files must be initialized.
- The OS handles all these duties for the user.



Services / Functions / Tasks of Operating System (OS)

3. Access to I/O devices (Resource allocation)

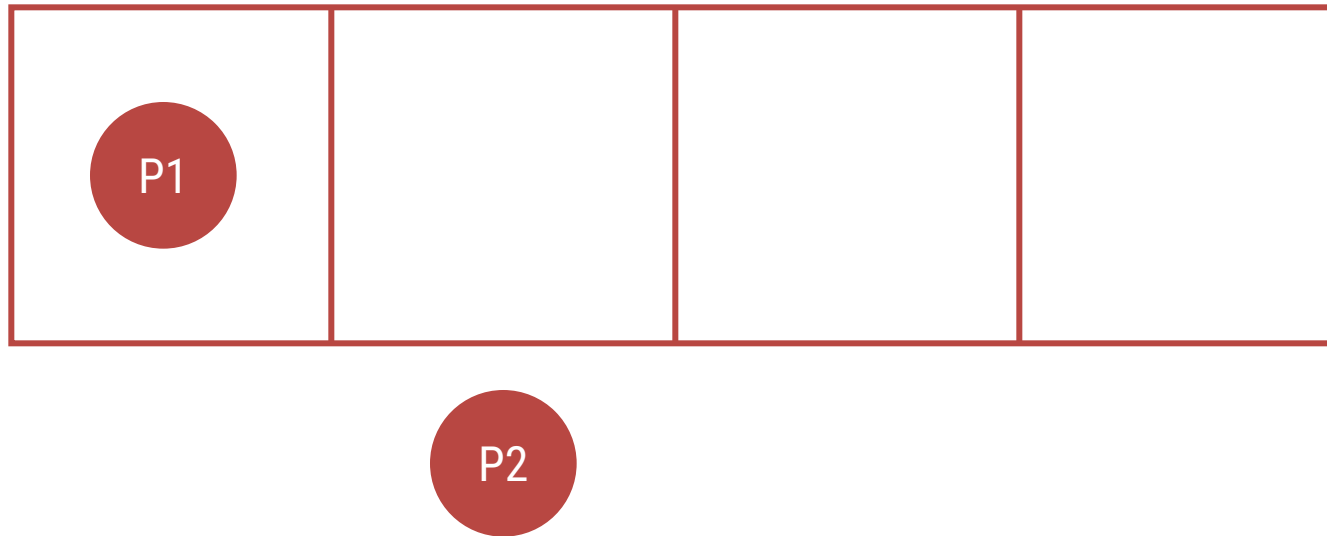
- ❑ A running program may require I/O, which may involve file or an I/O device.
- ❑ For efficiency and protection, users cannot control I/O devices directly.
- ❑ Therefore, the **OS controls these I/O devices** and **provides to program** as per requirement.



Services / Functions / Tasks of Operating System (OS)

4. Memory management

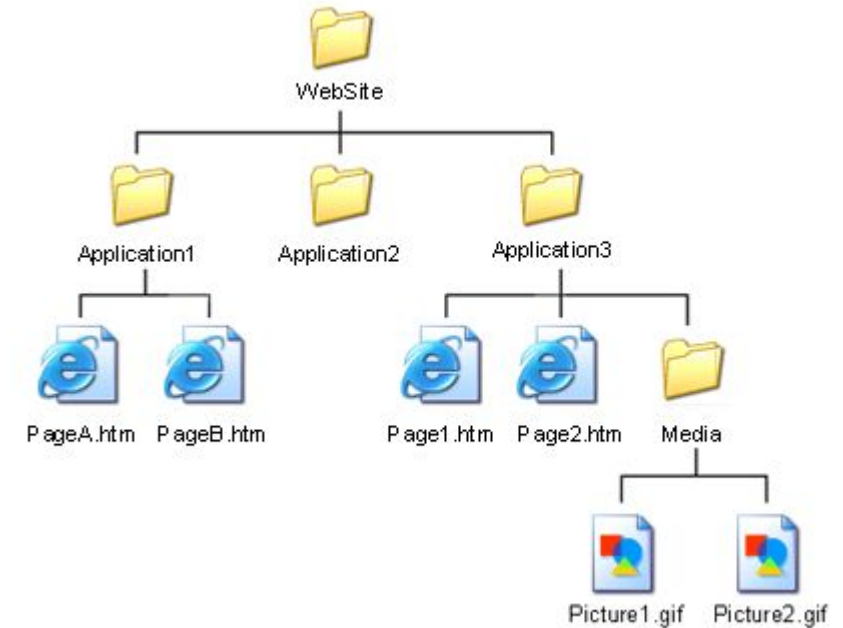
- ❑ OS **manages memory** hierarchy.
- ❑ OS **keeps the track** of which part of memory area in use and free memory.
- ❑ It **allocates memory** to program when they need it.
- ❑ It **de-allocate the memory** when the program finish execution.



Services / Functions / Tasks of Operating System (OS)

5. Controlled access to file

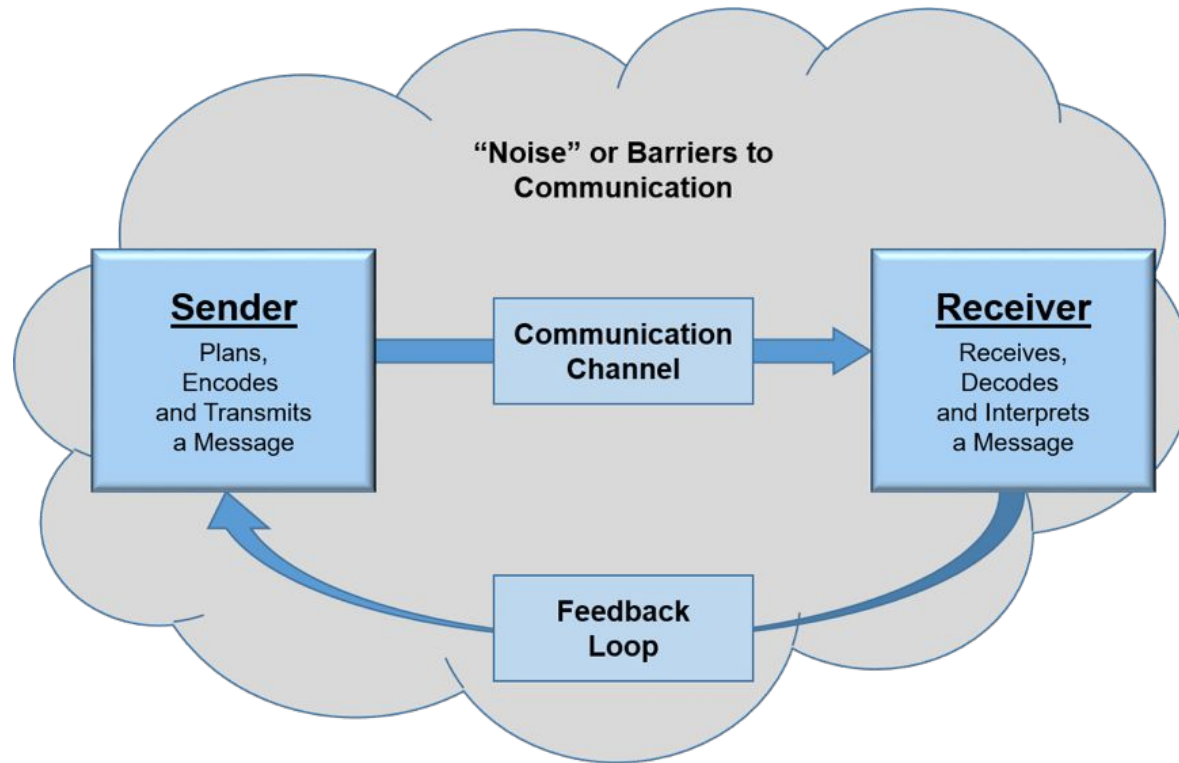
- In case of file access, OS provides a directory hierarchy for easy access and management of file.
- OS provides various file handling commands using which user can easily read, write and modify file.



Services / Functions / Tasks of Operating System (OS)

6. Communication

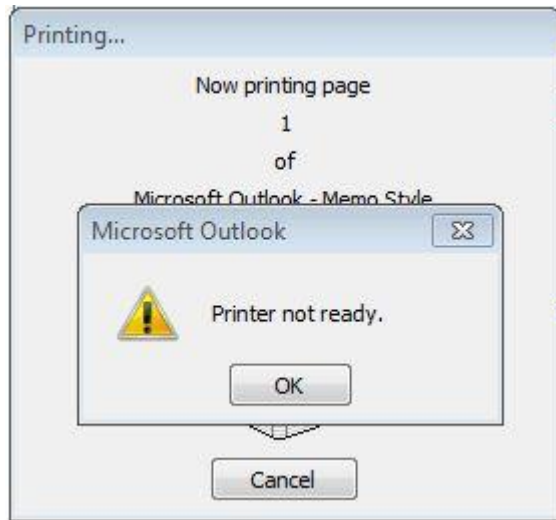
- ❑ In multitasking environment, the processes need to communicate with each other and to exchange their information.
- ❑ Operating system performs the communication among various types of processes in the form of shared memory.



Services / Functions / Tasks of Operating System (OS)

7. Error detection and response

- ❑ An error may occur in CPU, in I/O devices or in the memory hardware.
- ❑ Following are the major activities of an operating system with respect to error handling –
 - The OS **constantly checks for possible errors.**
 - The OS **takes an appropriate action** to ensure correct and consistent computing.



Services / Functions / Tasks of Operating System (OS)

9. Protection & Security

- Protection involves ensuring that **all accesses to system resources is controlled**.
- To **make a system secure**, the **user needs to authenticate himself or herself to the system**.





Types of Operating Systems (OS)

Section - 8

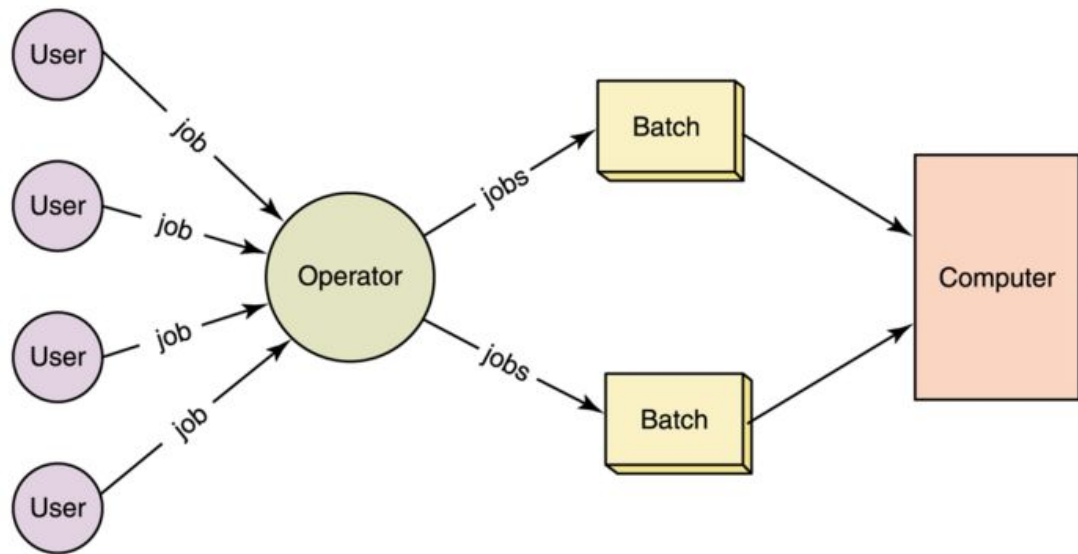
Types of Operating Systems (OS)

1. Mainframe operating systems

- OS found in room sized computers which are still found in major corporate data centers.
- They offer three kinds of services:
 1. Batch OS
 2. Transaction processing
 3. Timesharing
- Examples: OS/390, OS/360.

Mainframe Operating Systems services

Batch OS – processes routine jobs without any interactive user presents
i.e. claim processing in insurance



Transaction processing – handles large numbers of small processes
i.e. cheque processing at banks



Time Sharing Operating System

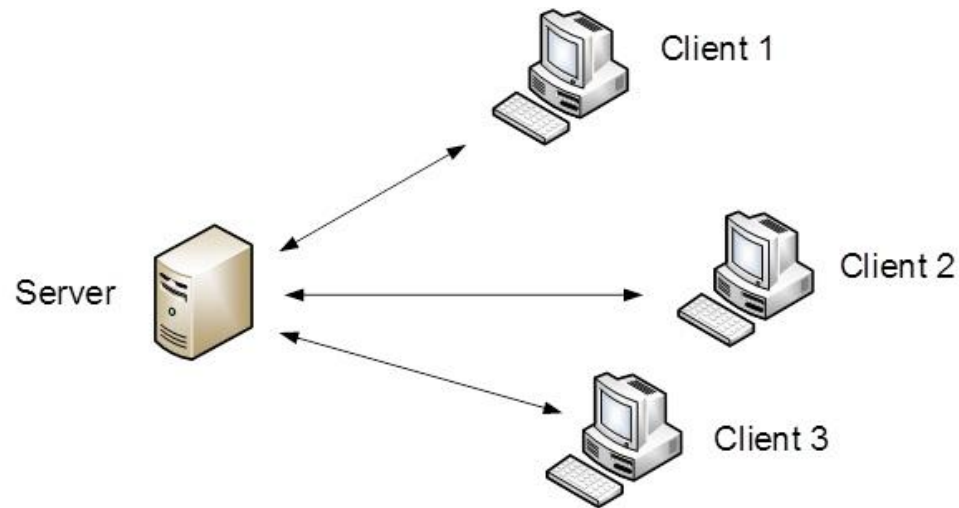


Timesharing – allows multiple remote users to run their jobs at once
i.e. querying a database, airline booking system

Types of Operating Systems (OS)

2. Server operating systems

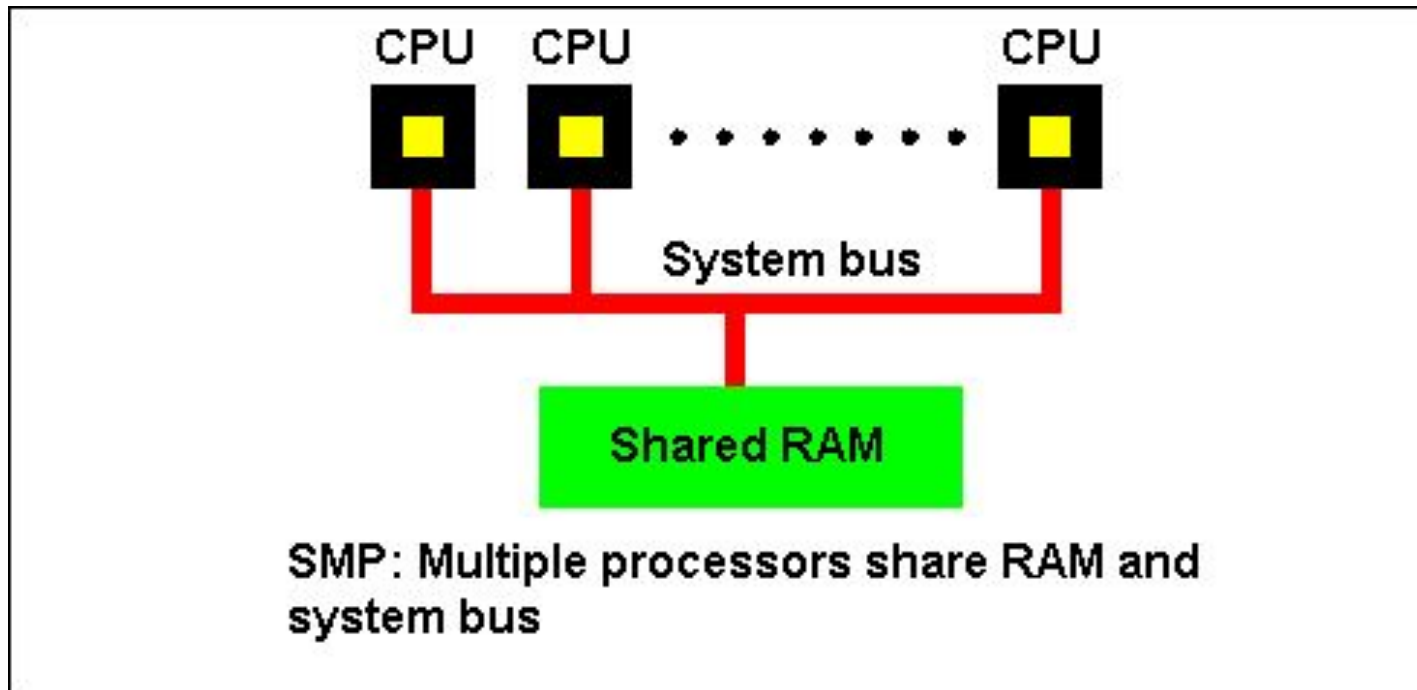
- This OS runs on servers which are very large PC, workstations or even mainframes.
- They serve multiple users at once over a network and allow the users to share hardware & software resources.
- It provides print services, file service or web service.
- It handles the incoming requests from clients.



Types of Operating Systems (OS)

3. Multiprocessor operating systems

- A computer system **consist two or more CPUs** is called multiprocessor.
- It is also called **parallel computers, multicomputer or multiprocessor**.
- Examples: Windows and Linux.



Types of Operating Systems (OS)

4. Personal computer operating systems

- ❑ The operating **systems installed on our personal computer and laptops** are personal OS.
- ❑ Job of this OS is to **provide good support to single user**.
- ❑ This OS is widely **used for word processing, spreadsheet and internet access**.
- ❑ Examples: Linux, Windows



Types of Operating Systems (OS)

5. Handhelds computer operating systems

- A handheld computer or PDA (Personal Digital Assistant) is **small computer that fit in a Pocket and perform small number of functions such as electronic address book, memo pad.**
- The OS runs on these devices are handheld OS.
- Examples: Symbian OS, Palm OS.



Types of Operating Systems (OS)

6. Embedded operating systems

- ❑ This OS is installed in **ATMs, printers, calculators and washing machine.**
- ❑ It runs on the computer that **control devices.**
- ❑ It **neither allow to download new software nor accept user installed software.** So there is no need for protection.



Types of Operating Systems (OS)

7. Smart card operating systems

- Smallest OS run on smart cards which are credit card sized devices containing CPU chip.
- These OS are installed on electronic payments cards such as debit card, credit card etc.
- They have limited processing power.





System calls

Section - 9

What is System calls?

- A system call allows **programs to interact with the operating system.**
- A system call is a **mechanism that provides the interface between a process and the operating system.**
- System call **provides the services of the operating system** to the user programs via **Application Program Interface(API).**
- System calls are the **only entry points for the kernel system.**operating system's services

Types of system calls

□ **Process Control:** This system calls perform the task of **process creation, process termination**, etc.

□ Functions:

- End and abort
- Load and execute
- Create process and terminate process
- Wait and signed event
- Allocate and free memory

□ **File Management:** File management system calls handle **file manipulation jobs like creating a file, reading, and writing**, etc.

□ Functions:

- Create a file
- Delete file
- Open and close file
- Read, write and reposition
- Get and set file attributes

Types of system calls

❑ **Device Management:** Device management does the job of **device manipulation like reading from device buffers, writing into device buffers**, etc.

❑ Functions

- Request and release device
- Logically attach/ detach devices
- Get and Set device attributes

❑ **Information Maintenance:** It **handles information** and its **transfer between the OS and user program**.

❑ Functions:

- Get or set time and date

Types of system calls

□ **Communication:** These types of system calls are specially used for **interprocess communications (IPC)**.

□ Functions:

- Create, delete communications connections
- Send, receive message
- Help OS to transfer status information
- Attach or detach remote devices

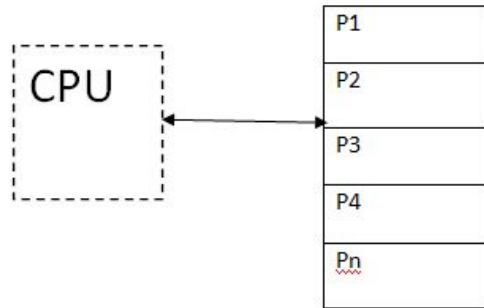


Multiprogramming v/s Multiprocessing v/s Multitasking

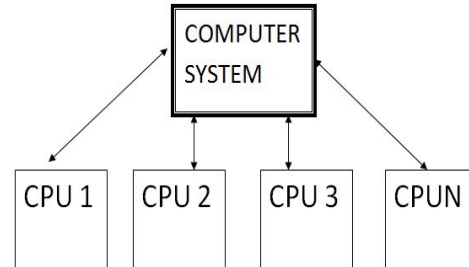
Section - 11

Multiprogramming v/s Multiprocessing v/s Multitasking

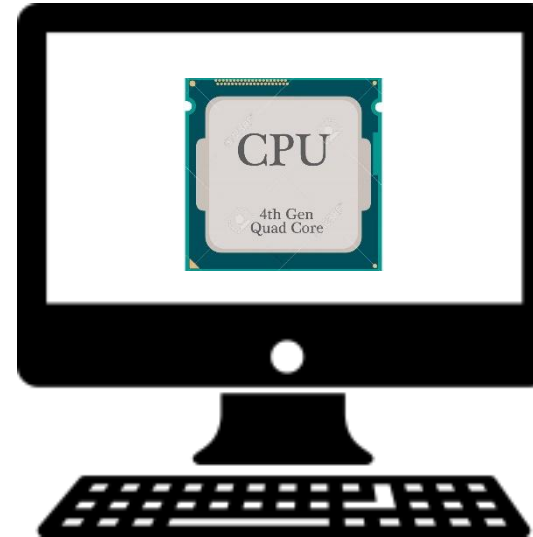
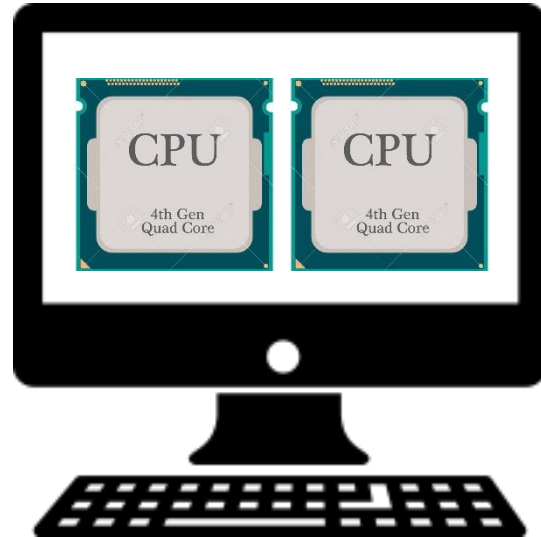
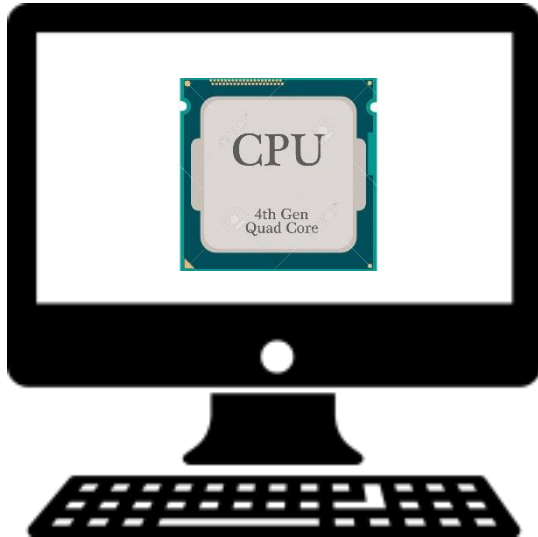
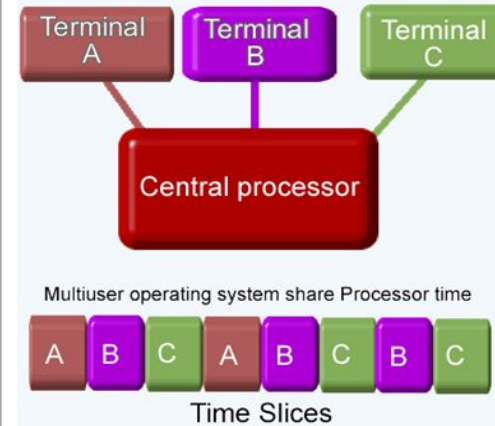
Multiprogramming



Multiprocessing



Multitasking



Multiprogramming v/s Multiprocessing v/s Multitasking

Multiprogramming	Multiprocessing	Multitasking
The concurrent residency of more than one program in the main memory is called as multiprogramming.	The availability of more than one processor per system, which can execute several set of instructions in parallel is called as multiprocessing	The execution of more than one task simultaneously is called as multitasking.
Number of processor: one	Number of processor: more than one	Number of processor: one
One process is executed at a time.	More than one process can be executed at a time.	One by one job is being executed at a time.

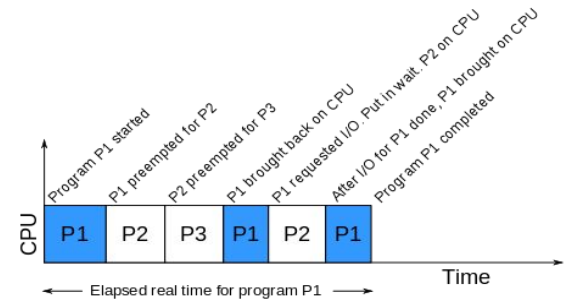
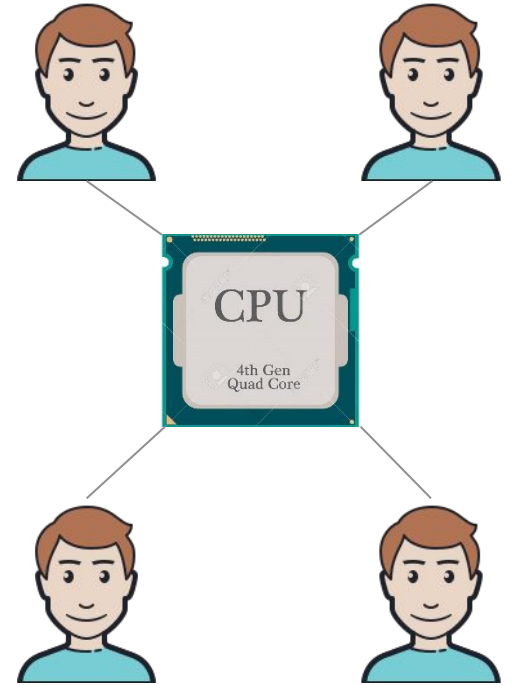


Time Sharing Operating System

Section - 12

Time Sharing Operating System

- ❑ A time sharing operating system **allows many users to share the computer resources simultaneously**.
- ❑ In other words, time sharing refers to the **allocation of computer resources in time slots to several programs simultaneously**.
- ❑ For example a **mainframe computer that has many users logged on to it**.
- ❑ Each user uses the resources of the mainframe i.e. memory, CPU etc.





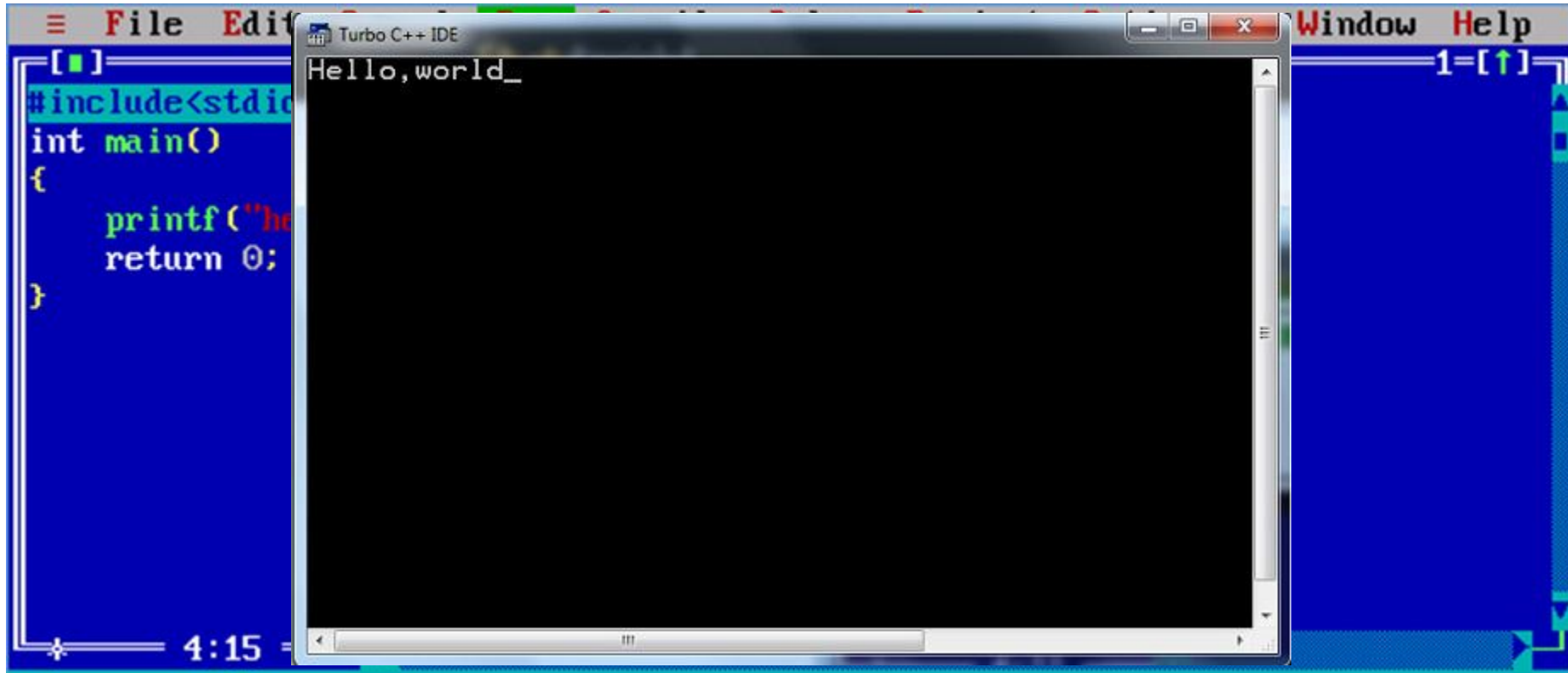
Process concept

Section - 13

What is Process?

Program

Process



What is Process?

- ❑ Process is a **program under execution**.
- ❑ Process is an **abstraction of a running program**.
- ❑ Process is an **instance of an executing program**, including the current values of the program counter, registers & variables.
- ❑ **Each process has its own virtual CPU.**



Process states

Section - 14

Process states



Running

Running:

Process is actually **using the CPU**



Ready

Ready:

Process is runnable, **temporarily stopped** to let another process to run



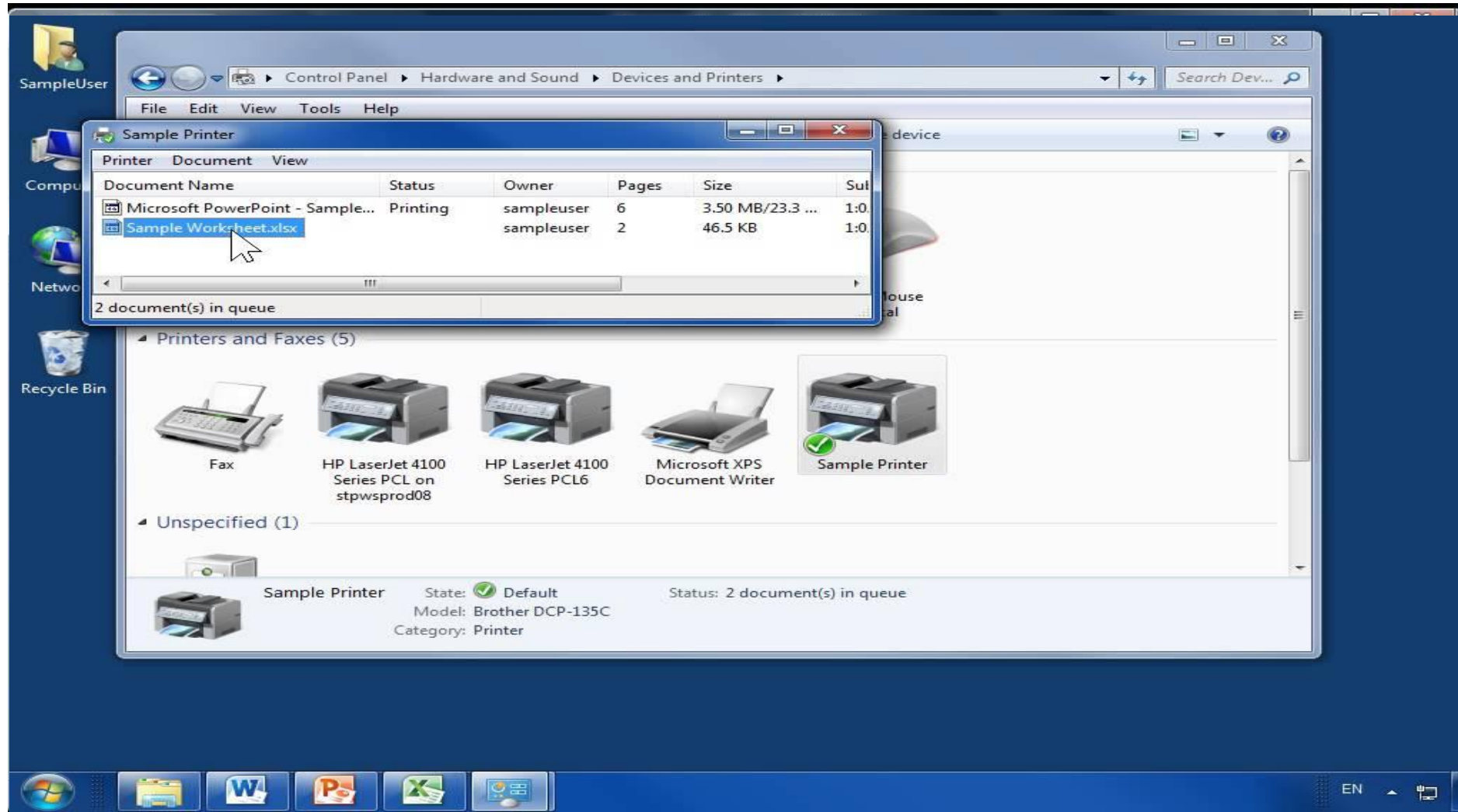
Blocked

Blocked:

Process is **unable to run** until some external event happens

Process states

Blocked



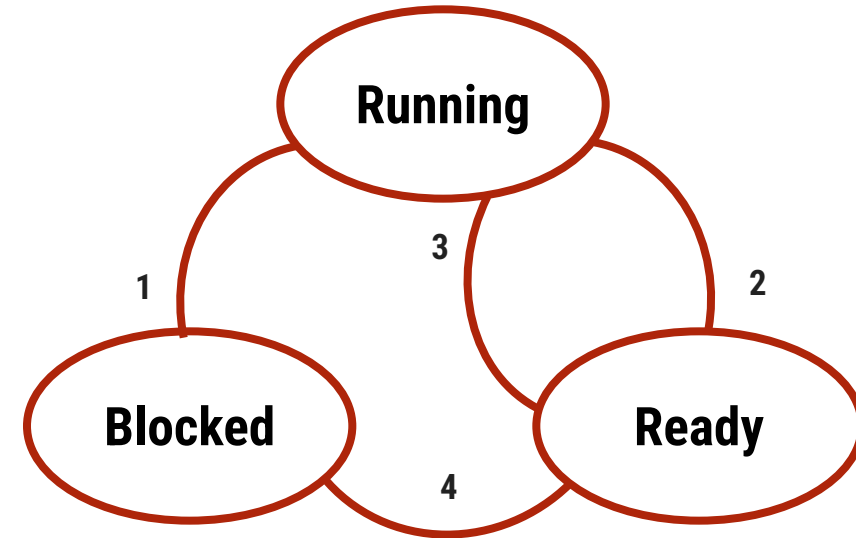


Process states transitions

Section - 15

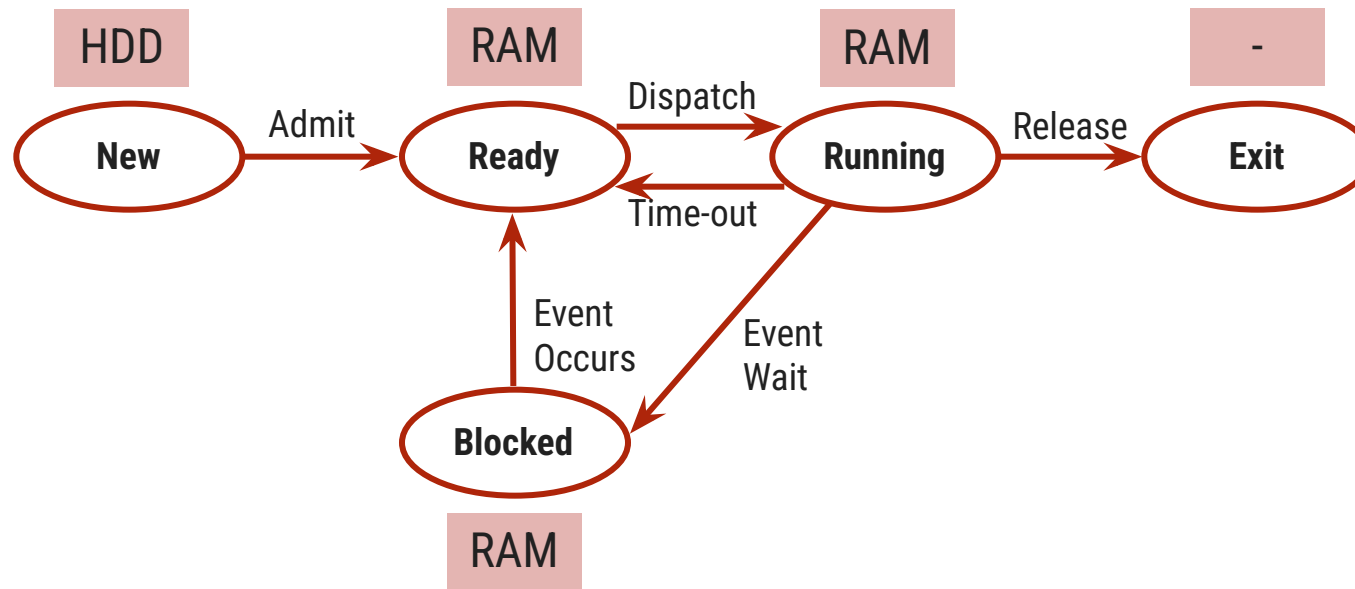
Process State Transitions

- When and how these transitions occur (process moves from one state to another)?
 1. **Process blocks for input or waits for an event** (i.e. printer is not available)
 2. Scheduler **picks another process**
 - End of time-slice or pre-emption.
 3. Scheduler **picks this process**
 4. **Input becomes available, event arrives** (i.e. printer become available)



Processes are always either executing (running) or waiting to execute (ready) or waiting for an event (blocked) to occur.

Five State Process Model and Transitions



- ❑ **New** – process is being **created**
- ❑ **Ready** – process is **waiting to run (runnable)**, temporarily stopped to let another process run
- ❑ **Running** – process is actually **using the CPU**
- ❑ **Blocked** – **unable to run** until some external event happens
- ❑ **Exit (Terminated)** – process has **finished the execution**

Exercise A process resides in which memory during different state?



Process Control Block (PCB)

Section - 16

What is Process Control Block (PCB)?

- ❑ A Process Control Block (PCB) is a **data structure maintained by the operating system for every process.**
- ❑ PCB is used for **storing the collection of information about the processes.**
- ❑ The PCB is **identified by an integer process ID (PID).**
- ❑ A PCB **keeps all the information needed to keep track of a process.**
- ❑ The PCB is maintained for a process **throughout its lifetime** and is **deleted once the process terminates.**
- ❑ The **architecture** of a PCB is completely **dependent on operating system** and may contain different information in different operating systems.
- ❑ PCB **lies in kernel memory space.**



Fields of Process Control Block (PCB)

- ❑ **Process ID** - Unique identification for each of the process in the operating system.
- ❑ **Process State** - The current state of the process i.e., whether it is ready, running, waiting.
- ❑ **Pointer** - A pointer to parent process.
- ❑ **Priority** - Priority of a process.
- ❑ **Program Counter** - Program Counter is a pointer to the address of the next instruction to be executed for this process.
- ❑ **CPU registers** - Various CPU registers where process need to be stored for execution for running state.
- ❑ **IO status information** - This includes a list of I/O devices allocated to the process.
- ❑ **Accounting information** - This includes the amount of CPU used for process execution, time limits etc.

Process ID
State
Pointer
Priority
Program counter
CPU registers
I/O information
Accounting information
etc....



What is process scheduling?

Section - 17

What is process scheduling?

- ❑ Process scheduling is the **activity of the process manager** that handles **suspension of running process from CPU and selection of another process** on the basis of a particular strategy.
- ❑ The **part of operating system** that **makes the choice** is called **scheduler**.
- ❑ The **algorithm used by** this **scheduler** is called scheduling **algorithm**.
- ❑ Process scheduling is an essential part of a multiprogramming operating systems.



Objectives (goals) of scheduling

- ❑ **Fairness**: giving each process a fair share of the CPU.
- ❑ **Balance**: keeping all the parts of the system busy (**Maximize**).
- ❑ **Throughput**: no of processes that are completed per time unit (**Maximize**).
- ❑ **Turnaround time**: time to execute a process from submission to completion (**Minimize**).
 - ❑ **Turnaround time** = Process finish time – Process arrival time
- ❑ **CPU utilization**: percent of time that the CPU is busy in executing a process.
 - ❑ keep CPU as busy as possible (**Maximized**).
- ❑ **Response time**: time between issuing a command and getting the result (**Minimized**).
- ❑ **Waiting time**: amount of time a process has been waiting in the ready queue (**Minimize**).
 - ❑ **Waiting time** = Turnaround time – Actual execution time

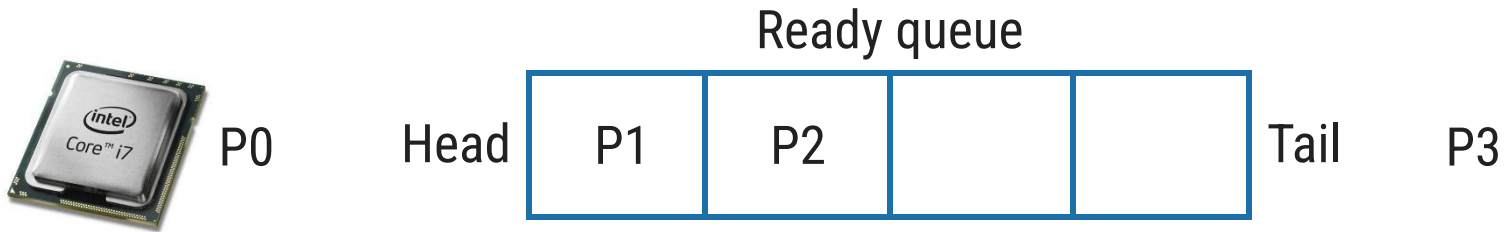
Scheduling algorithms

1. First Come First Served (FCFS)
2. Shortest Job First (SJF)
3. Shortest Remaining Time Next (SRTN)
4. Round Robin (RR)
5. Priority
 - I. Preemptive
 - II. Non-Preemptive

First Come First Served (FCFS)

□ Selection criteria:

- The **process that request first is served first**.
- It means that **processes are served in the exact order of their arrival**.



□ Decision Mode:

- **Non preemptive:** Once a process is selected, it runs until it is blocked for an I/O or some other event or it is terminated.

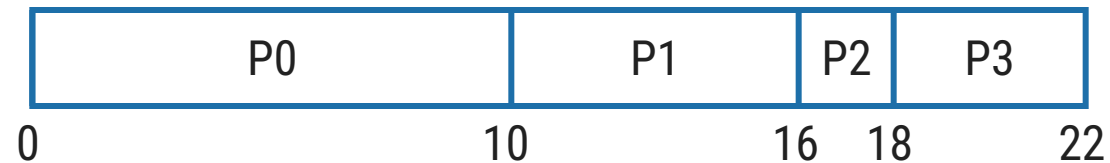
□ Implementation:

- This strategy can be easily **implemented by using FIFO** (First In First Out) queue.
- When CPU becomes free, a process from the first position in a queue is selected to run.

First Come First Served (FCFS)

Process	Arrival Time (T0)	Burst Time (ΔT)	Finish Time (T1)	Turnaround Time (TAT = T1 - T0)	Waiting Time (WT = TAT - ΔT)
P0	0	10	10	10	0
P1	1	6	16	15	9
P2	3	2	18	15	13
P3	5	4	22	17	13

□ Gantt Chart



□ Average Turnaround Time: $(10+15+15+17)/4 = 14.25$ ms.

□ Average Waiting Time: $(0+9+13+13)/4 = 8.75$ ms.

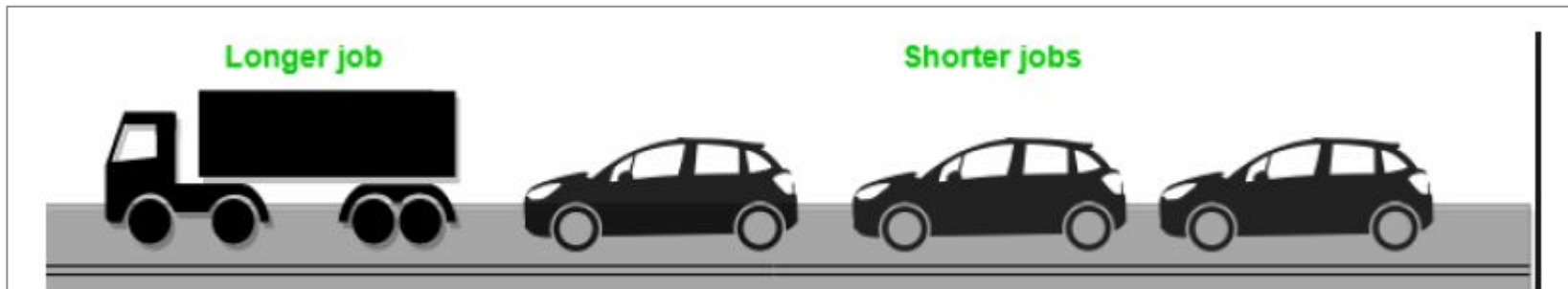
First Come First Served (FCFS)

□ Advantages

- **Simple** and **fair**.
- **Easy to understand** and **implement**.
- Every process will get a chance to run, so **starvation doesn't occur**.
 - Starvation is the **problem that occurs when high priority processes keep executing and low priority processes get blocked for indefinite time**.

□ Disadvantages

- **Not efficient** because average waiting time is too high.
- **Convoy effect is possible**. All small I/O bound processes wait for one big CPU bound process to acquire CPU.

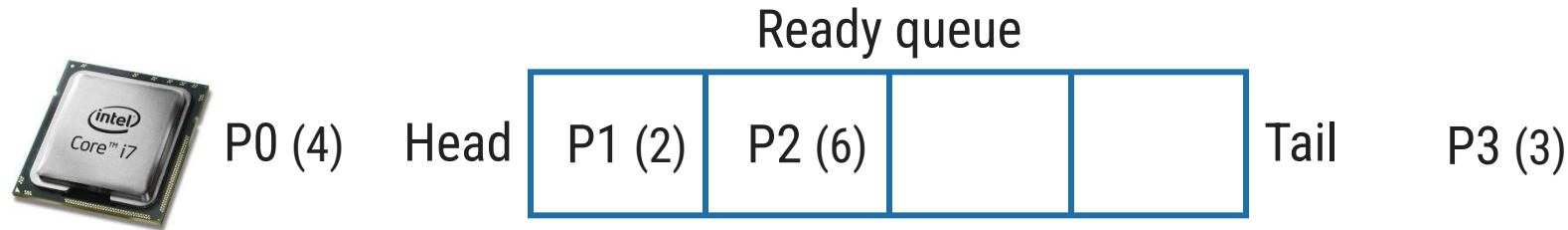


- **CPU utilization may be less efficient** especially when a CPU bound process is running with many I/O bound processes.

Shortest Job First (SJF)

□ Selection criteria:

- The process, that **requires shortest time to complete execution, is served first.**



□ Decision Mode:

- **Non preemptive:** Once a process is selected, it runs until it is blocked for an I/O or some other event or it is terminated.

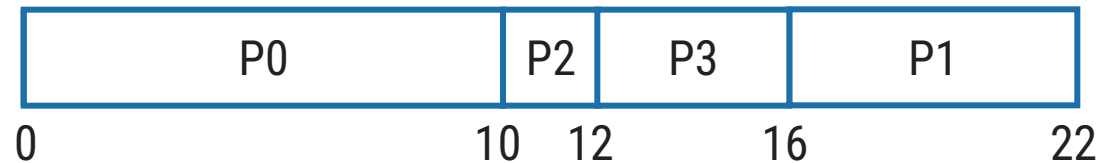
□ Implementation:

- This strategy can be easily **implemented by using sorted FIFO** (First In First Out) queue.
- All processes in a queue are **sorted in ascending order based on their required CPU bursts.**
- When CPU becomes free, a process from the first position in a queue is selected to run.

Shortest Job First (SJF)

Process	Arrival Time (T ₀)	Burst Time (ΔT)	Finish Time (T ₁)	Turnaround Time (TAT = T ₁ - T ₀)	Waiting Time (WT = TAT - ΔT)
P0	0	10	10	10	0
P1	1	6	22	21	15
P2	3	2	12	9	7
P3	5	4	16	11	7

□ Gantt Chart



□ Average Turnaround Time: $(10+21+9+11)/4 = 12.75$ ms.

□ Average Waiting Time: $(0+15+7+7)/4 = 7.25$ ms.

Shortest Job First (SJF)

□ Advantages

- **Less waiting time.**
- **Good response for short processes.**

□ Disadvantages

- It is **difficult to estimate time required** to complete execution.
 - **Starvation is possible for long process.** Long process may wait forever.
 - Starvation is the problem that occurs when high priority processes keep executing and low priority processes get blocked for indefinite time.
-

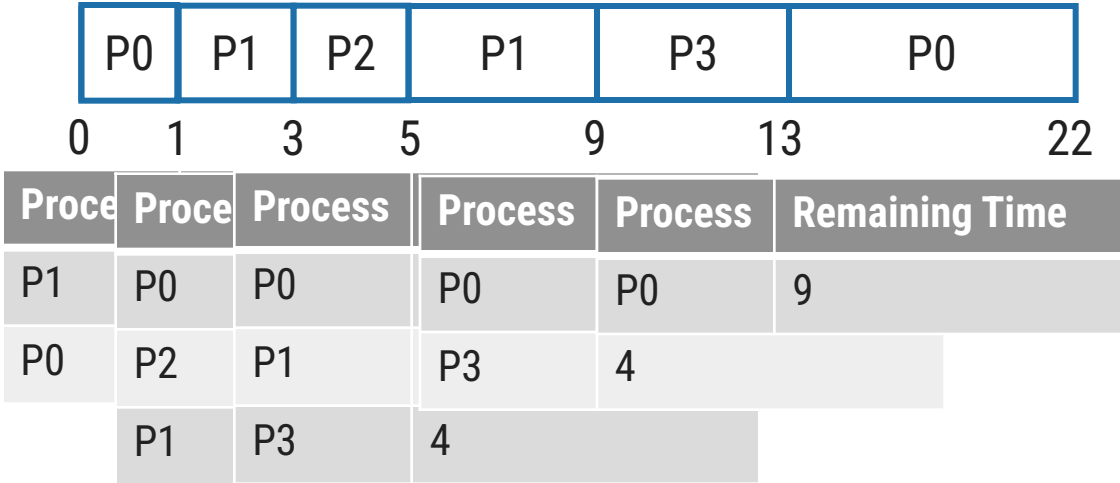
Shortest Remaining Time Next (SRTN)

Process	Arrival Time (T0)	Burst Time (ΔT)	Finish Time (T1)	Turnaround Time (TAT = T1 - T0)	Waiting Time (WT = TAT - ΔT)
P0	0	10	22	22	12
P1	1	6	9	8	2
P2	3	2	5	2	0
P3	5	4	13	8	4

□ Gantt Chart

□ Average Turnaround Time:

□ Average Waiting Time:



Shortest Remaining Time Next (SRTN)

□ Advantages

- **Less waiting time.**
- **Quite good response for short processes.**

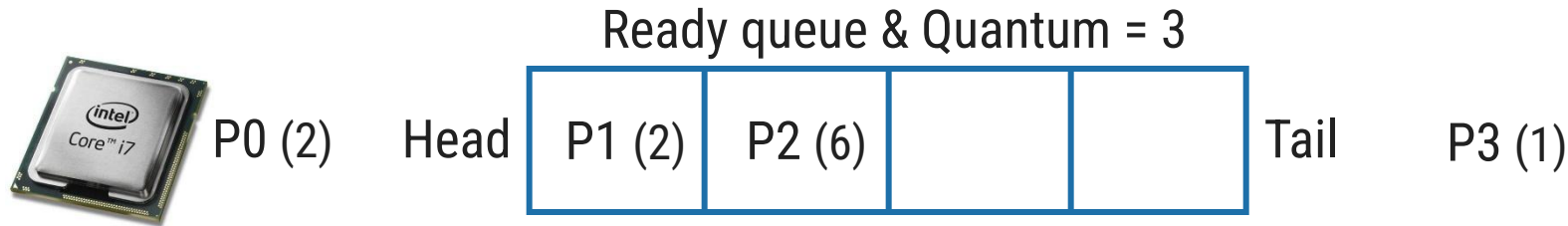
□ Disadvantages

- It is **difficult to estimate time required** to complete execution.
- **Starvation is possible for long process.** Long process may wait forever.
 - Starvation is the problem that occurs when high priority processes keep executing and low priority processes get blocked for indefinite time.
- **Context switch overhead is there.**

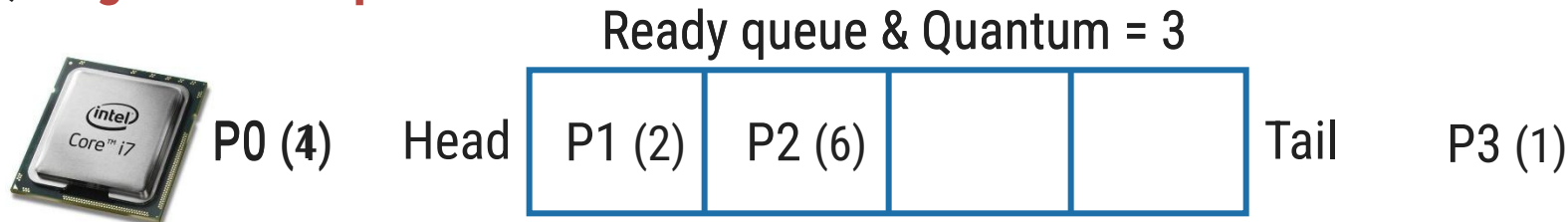
Round Robin (RR)

□ Selection criteria:

- Each selected process is **assigned a time interval, called time quantum or time slice.**
- Process is **allowed to run only for this time interval.**
- Here, two things are possible:
- First, **process is either blocked or terminated before the quantum has elapsed.** In this case the **CPU switching is done** and **another process is scheduled** to run.



- Second, **process needs CPU burst longer than time quantum.** In this case, process is **running at the end of the time quantum.**
- Now, it will be **preempted** and **moved to the end of the queue.**
- CPU will be **allocated to another process.**
- Here, **length of time quantum is critical to determine.**



Round Robin (RR)

□ Decision Mode:

- **Preemptive:** When a new process arrives, its total time is compared to the current process remaining run time.
- **Selection of new job is as per FCFS** scheduling algorithm.

□ Implementation:

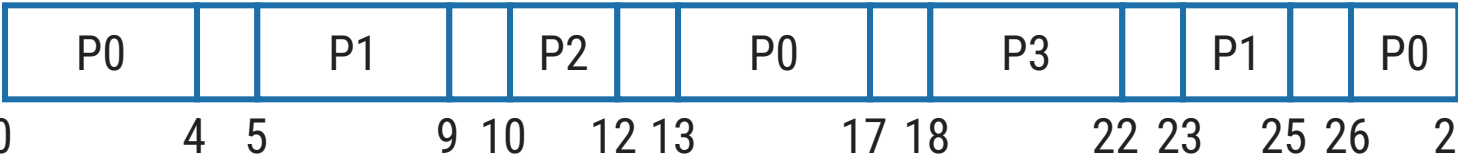
- This strategy can be implemented by using **circular FIFO queue**.
- If any process comes, or process releases CPU, or process is preempted. It is moved to the end of the queue.
- When CPU becomes free, a process from the first position in a queue is selected to run.

Round Robin (RR)

Process	Arrival Time (T0)	Burst Time (ΔT)	Finish Time (T1)	Turnaround Time (TAT = T1 - T0)	Waiting Time (WT = TAT - ΔT)
P0	0	10	28	28	18
P1	1	6	25	24	18
P2	3	2	12	9	7
P3	5	4	22	17	13

Gantt Chart

- Quantum time is 4 ms
- Context switch overhead



Avg. Turnaround Time

Avg. Waiting Time:

Process	Remaining Time	Process	Remaining Time	Process	Remaining Time	Process	Remaining Time	Process	Remaining Time
P1	6	P2	2	P0	6	P3	4	P1	2
P2	2	P0	2	P3	2	P1	2	P0	2
P0	2	P3	2	P1	2	P0	2		
		P1	2						

Round Robin (RR)

□ Advantages

- **Simplest, fairest and most widely used algorithms.**

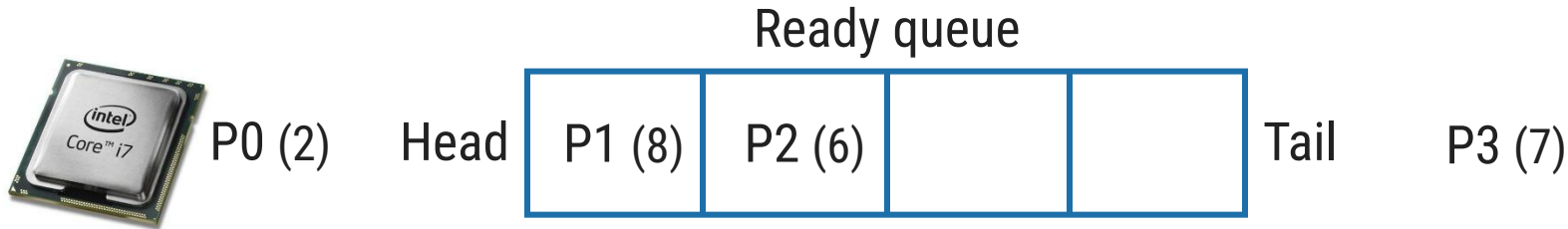
□ Disadvantages

- **Context switch overhead is there.**
 - **Determination of time quantum is too critical.**
 - If it is **too short**, it causes frequent context switches and lowers CPU efficiency.
 - If it is **too long**, it causes poor response for short interactive process.
-

Non-Preemptive Priority

□ Selection criteria:

- The process, that **has highest priority, is served first.**



□ Decision Mode:

- **Non preemptive:** Once a process is selected, it runs until it is blocked for an I/O or some other event or it is terminated.

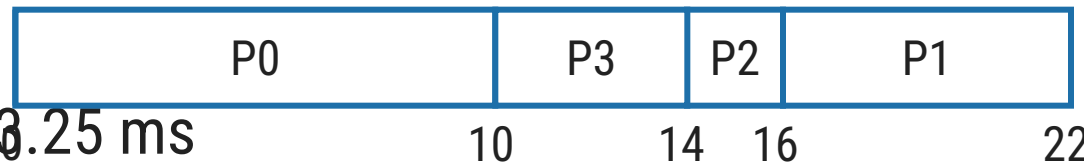
□ Implementation:

- This strategy can also be implemented by using **sorted FIFO queue.**
- All processes in a queue are **sorted based on their priority with highest priority process at front end.**
- When CPU becomes free, a process from the first position in a queue is selected to run.

Non-Preemptive Priority

Process	Arrival Time (T0)	Burst Time (ΔT)	Priority	Finish Time (T1)	Turnaround Time (TAT = T1 - T0)	Waiting Time (WT = TAT - ΔT)
P0	0	10	5	10	10	0
P1	1	6	4	22	21	15
P2	3	2	2	16	13	11
P3	5	4	0	14	9	5

□ Gantt Chart (small values for priority means higher priority of a process)



□ Avg. Turnaround Time: 13.25 ms

□ Avg. Waiting Time: 7.75 ms

Non-Preemptive Priority

□ Advantages

- **Priority is considered so critical processes can get even better response time.**

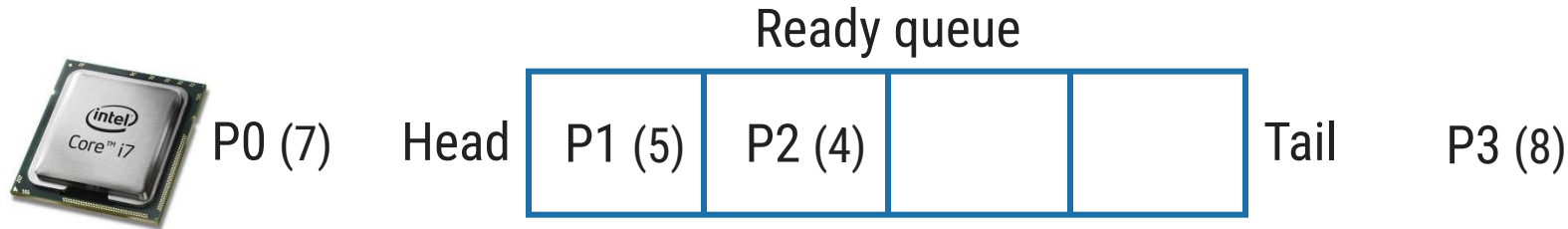
□ Disadvantages

- **Starvation is possible for low priority processes.** It can be overcome by using technique called '**Aging**'.
 - **Aging: gradually increases the priority of processes that wait in the system for a long time.**
-

Preemptive Priority

□ Selection criteria:

- The process, that **has highest priority, is served first.**



□ Decision Mode:

- **Preemptive:** When a new process arrives, its priority is compared with current process priority.
- If the **new process has higher priority than the current, the current process is suspended and new job is started.**

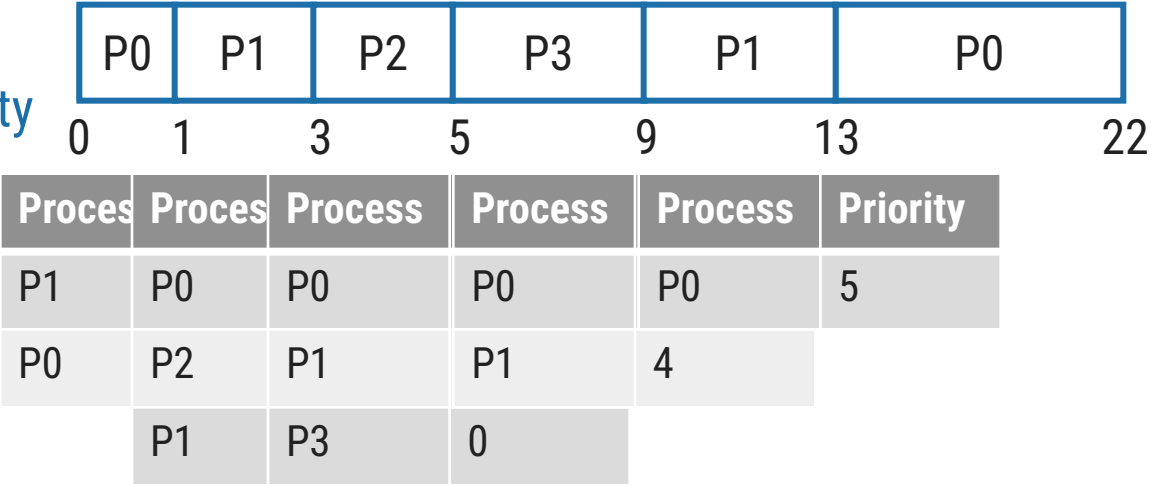
□ Implementation:

- This strategy can also be implemented by using **sorted FIFO queue.**
- All processes in a queue are **sorted based on their priority with highest priority process at front end.**
- When CPU becomes free, a process from the first position in a queue is selected to run.

Preemptive Priority

Process	Arrival Time (T0)	Burst Time (ΔT)	Priority	Finish Time (T1)	Turnaround Time (TAT = T1 - T0)	Waiting Time (WT = TAT - ΔT)
P0	0	10	5	22	22	12
P1	1	6	4	13	12	6
P2	3	2	2	5	2	0
P3	5	4	0	9	4	0

- Gantt Chart
 - small values means higher priority
- Avg. Turnaround Time: 10 ms
- Avg. Waiting Time: 4.5 ms



Preemptive Priority

□ Advantages

- **Priority is considered so critical processes can get even better response time.**

□ Disadvantages

- **Starvation is possible for low priority processes.** It can be overcome by using technique called '**Aging**'.
 - **Aging: gradually increases the priority of processes that wait in the system for a long time.**
 - **Context switch overhead is there.**
-



***Thank
You***

